



ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

(наименование образовательного учреждения)

ДОПУСКАЮ К ЗАЩИТЕ

Заместитель директора по УПР *О.В.Корешков*

(дата)

ДИПЛОМНАЯ РАБОТА

Разработка мобильного приложения для бронирования билетов на
массовые мероприятия

(тема)

выполнена

студентом группы

3 ИСП11-26

(номер группы)

Рязановой Юлией Романовной

(И.О.Фамилия)

(подпись, дата)

Основная профессиональная образовательная программа по специальности
09.02.07 Информационные системы и программирование

(шифр и наименование специальности)

Форма обучения

очная

Руководитель

преподаватель

Кумскова

Ирина

Александровна

(ученая степень, должность,

И.О.Фамилия)

(подпись, дата)

Руководитель

преподаватель

Прилепская

Татьяна

Владимировна

(ученая степень, должность,

И.О.Фамилия)

(подпись, дата)

Председатель предметной (междисциплинарной, модульной) комиссии

Юлия Васильевна Климова

(И.О.Фамилия)

(подпись, дата)

Москва
2025



ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

(наименование образовательного учреждения)

УТВЕРЖДАЮ

Заместитель директора по УПР *О.В.Корешков*

(дата)

ЗАДАНИЕ НА ДИПЛОМНУЮ РАБОТУ

Студенту *Рязановой Юлии Романовне*

(фамилия, имя, отчество полностью)

I. Тема дипломной работы

Разработка мобильного приложения для бронирования билетов на массовые мероприятия

II. Срок сдачи студентом законченной работы

«7» июня 2025 г.

III. Исходные данные

*Отчет по преддипломной практике, нормативно-правовые документы
по разработке программного обеспечения, документация предприятия -
базы практики*

IV. Перечень подлежащих разработке вопросов

- Аналитический обзор литературных источников по тематике дипломной работы*
- Сбор и структуризация информации по результатам обследования предметной области*
- Проектирование мобильного приложения*
- Проектирование и создание базы данных*
- Реализация и отладка программы в выбранной среде разработки*
- Тестирование программы и разработка документации*

V. Перечень графического/иллюстрационного материала

- Презентация, 15 слайдов*
-
-
-
-

VI. Дата выдачи задания

«21» апреля 2025 г.

Руководитель

И.А.Кумскова

(И.О.Фамилия)

(подпись)

Задание принял к
исполнению

(И.О.Фамилия)

(подпись)

«21» апреля 2025 г.



ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

КАЛЕНДАРНЫЙ ПЛАН ДИПЛОМНОЙ РАБОТЫ
(с указанием сроков выполнения отдельных этапов)

для специальности 09.02.07 Информационные системы и программирование
(группа ЗИСП11-26)

Сроки преддипломной практики		4 недели с 21.04.25 по 17.05.25 Согласно указу Президента РФ от 23 апреля 2021 г. № 242
	Выбор темы, руководителя, оформление заявления	с 21.04.25 по 26.04.25
	Утверждение темы дипломной работы	28.04.25
	Выполнение задания по теме дипломной работы	с 28.04.25 по 08.05.25
	Предоставление отчета по практике руководителю	с 12.05.25 по 17.05.25
	Аттестация по практике	17.05.25
Подготовка ВКР		4 недели с 19.05.25 по 14.06.25 согласно учебному плану
	Утверждение задания на дипломную работу	1 неделя с 19.05.25 по 25.05.25
	Подбор и анализ исходной информации	
	Подготовка и утверждение плана (оглавления) дипломной работы	с 19.05.25 по 25.05.25
	Работа над разделами (главами) и устранение замечаний руководителя дипломной работы	1 неделя с 26.05.25 по 31.05.25
	Согласование содержания дипломной работы, устранение замечаний	1 неделя с 02.06.25 по 06.06.25
	Оформление и представление руководителю полного текста работы. Получение отзыва руководителя дипломной работы.	06.06.25
	Предоставление студентом готовой дипломной работы рецензенту	07.06.25
	Сдача ДЭ	Согласно отдельному графику
	Предзащита дипломной работы	09.06.25-13.06.25

Руководитель _____ И.О. Фамилия

План принял к исполнению «__» _____ 2025 г.
_____ (_____)

Оглавление

Введение	5
Глава 1. Аналитическая часть.....	7
1.1 Анализ предметной области	7
1.2 Определение требований	11
1.3 Анализ существующих решений на рынке программного обеспечения	15
1.4. Составление проекта управлением разработкой ИС	17
1.5. Разработка технического задания (Приложение 1)	19
1.6. Проектирование информационной системы.....	19
1.7. Проектирование базы данных.....	25
1.8. Проектирование интерфейса ИС	27
Вывод по теоретической части	32
Глава 2. Практическая часть	33
2.1. Создание базы данных	33
2.2. Разработка клиентской части.....	35
2.4 Разработка руководства пользователя (Приложение №3)	40
Вывод по практической части	40
Заключение	41
Список использованных источников	42
Приложения	44
ПРИЛОЖЕНИЕ №1	44
ПРИЛОЖЕНИЕ №2	49
ПРИЛОЖЕНИЕ №3	57

Введение

Современные пользователи всё чаще предпочитают решать свои задачи с помощью мобильных устройств. Это относится и к сфере развлечений, где удобство и скорость доступа к информации играют ключевую роль. Процесс бронирования билетов на массовые мероприятия, включающий в себя поиск, выбор и оплату, должен быть максимально простым и интуитивно понятным. Успешная организация этого процесса напрямую влияет на удовлетворённость клиентов и прибыль организаторов мероприятий. Грамотно спроектированное мобильное приложение может значительно улучшить пользовательский опыт и оптимизировать процессы бронирования. При организации процесса бронирования билетов через мобильное приложение необходимо решить следующие задачи:

- Обеспечение быстрого и удобного поиска мероприятий и доступных билетов.
- Четкая и понятная навигация по приложению.
- Минимизация количества шагов, необходимых для бронирования и оплаты билета.
- Обеспечение безопасности транзакций и защиты персональных данных пользователей.
- Контроль статуса заказа и предоставление своевременной информации о подтверждении и отправке билетов.

Процесс бронирования должен быть максимально простым и интуитивно понятным, занимать минимум времени и не вызывать затруднений у пользователей.

Актуальность дипломной работы обусловлена растущей популярностью мобильных приложений и необходимостью предоставления удобного и эффективного инструмента для бронирования билетов на массовые мероприятия, особенно для организаторов, стремящихся расширить свою

аудиторию и повысить продажи. Многие существующие приложения либо не обладают достаточной функциональностью, либо неудобны в использовании.

Целью разработки и внедрения мобильного приложения для бронирования билетов является повышение удобства и эффективности процесса покупки билетов для пользователей, а также увеличение продаж и оптимизация работы организаторов мероприятий. Приложение должно обеспечивать быстрый поиск, удобный выбор, безопасную оплату и оперативное получение билетов.

Задачи дипломной работы:

- Анализ предметной области и проектирование системы бронирования билетов.
- Разработка технического задания на мобильное приложение.
- Создание базы данных для хранения информации о мероприятиях, билетах и пользователях.
- Разработка мобильного приложения и его пользовательского интерфейса.

Объектом дипломной работы является процесс бронирования билетов на массовые мероприятия. Предмет исследования – система бронирования билетов, реализованная в виде мобильного приложения.

Дипломная работа состоит из двух частей. Первая часть включает в себя анализ предметной области, определение требований к информационной системе, анализ существующих мобильных приложений для бронирования билетов, разработка технического задания и проектирование информационной системы, базы данных и пользовательского интерфейса. Вторая часть включает в себя реализацию разработанных в первой главе моделей данных и информационной системы, тестирование приложения и написание руководства пользователя.

Глава 1. Аналитическая часть

1.1 Анализ предметной области

Предметная область данной дипломной работы – бронирование билетов на массовые мероприятия с использованием мобильного приложения. Эта область охватывает процессы поиска, выбора, бронирования и оплаты билетов на различные события, такие как фестивали, спортивные соревнования, и другие. Успешное функционирование в этой области требует эффективного взаимодействия между пользователями, желающими приобрести билеты, и платформой, обеспечивающей процесс бронирования.

Основной вызов в этой сфере – предоставление удобного, быстрого и надежного способа приобретения билетов в условиях высокой конкуренции и быстро меняющихся предпочтений пользователей. Эффективность приложения для бронирования билетов напрямую влияет на удовлетворенность клиентов, увеличение продаж и снижение затрат организаторов мероприятий.

Анализ предметной области включает в себя рассмотрение различных аспектов, таких как:

- Пользователи: Потенциальные покупатели билетов, их потребности, предпочтения, техническая грамотность, готовность использовать мобильные приложения.
- Мероприятия: Разнообразие форматов, масштабов, ценовых категорий, целевой аудитории.
- Билеты: Различные типы билетов, цены, условия приобретения и возврата.
- Процесс бронирования: Этапы поиска, выбора, оформления заказа, оплаты и получения билета.
- Технологии: Мобильные платформы (Android), базы данных, платежные системы, системы безопасности.

- Конкуренты: Анализ существующих мобильных приложений и веб-сервисов для бронирования билетов, их сильные и слабые стороны.
- Законодательство: Правила продажи билетов, защита прав потребителей, обработка персональных данных.

Основной проблемой в сфере бронирования билетов является необходимость обеспечения удобного и безопасного доступа к информации о мероприятиях и возможности приобретения билетов в любое время и в любом месте. Традиционные методы бронирования, такие как посещение касс или использование веб-сайтов, часто являются неудобными и требуют значительных временных затрат. Мобильное приложение позволяет решить эту проблему, предоставляя пользователям возможность быстро и легко находить и приобретать билеты с помощью своих смартфонов.

Для создания функциональной модели системы, отображающей структуру и функции приложения, а также потоки информации между ними, будет использована нотация графического моделирования IDEF0. Эта методология позволяет представить систему в виде набора взаимосвязанных блоков, отражающих процессы и операции, и поддерживает последовательную декомпозицию каждого процесса до необходимого уровня детализации.

Для более глубокого понимания процессов, происходящих в предметной области бронирования билетов на массовые мероприятия, необходимо декомпонировать их на отдельные этапы и представить их в виде диаграмм. Использование графических нотаций позволяет наглядно представить взаимодействие пользователей, системы и внешних факторов.

На рисунке 1.1 представлена контекстная диаграмма системы бронирования билетов, которая определяет границы и взаимодействие системы с внешним миром.

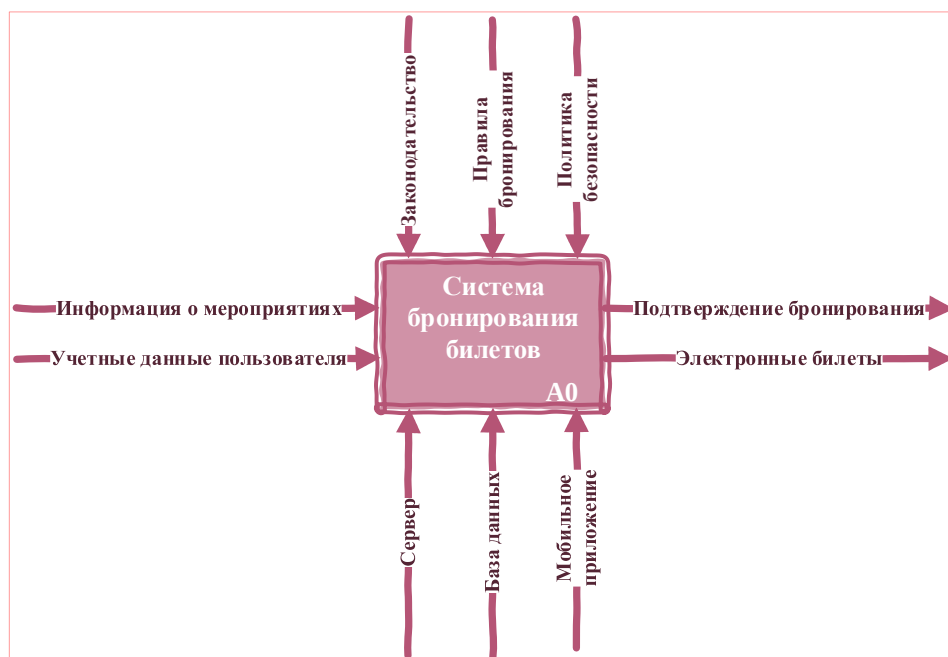


Рисунок 1.1 Контекстная диаграмма системы бронирования билетов

Декомпозиция общего блока на связанные между собой элементы представлена на рисунке 1.2.

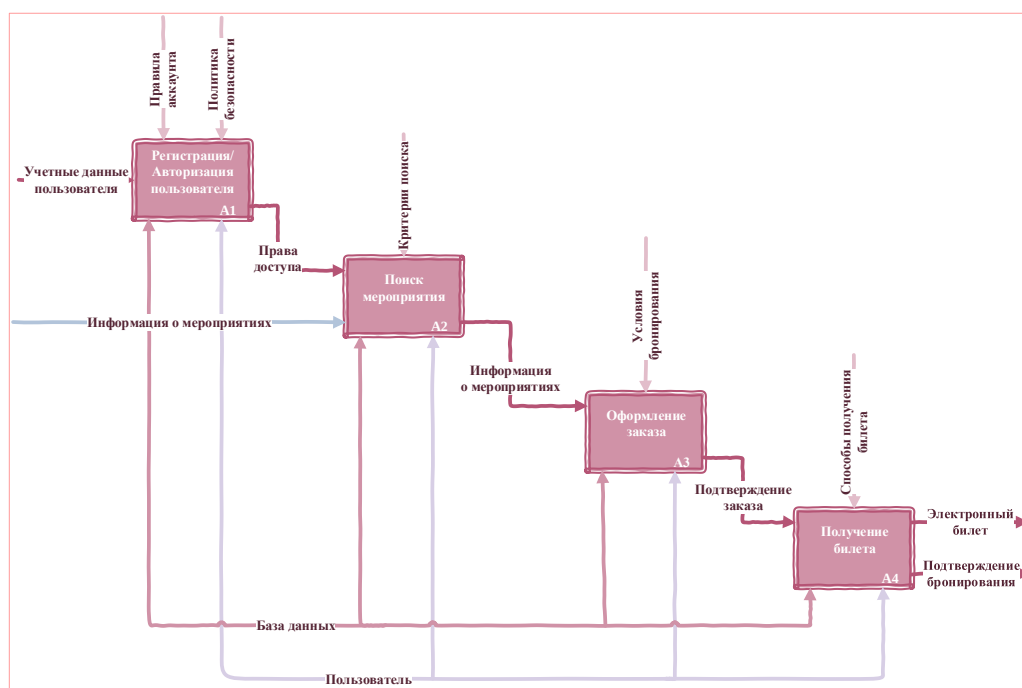


Рисунок 1.2 Декомпозиция первого уровня

Каждую из этих функций можно детализировать, разбив её на составные элементы с помощью подчинённой диаграммы более низкого уровня. В

качестве примера, на рисунке 1.3 подробно описан процесс A2, отвечающий за "Поиск мероприятия".

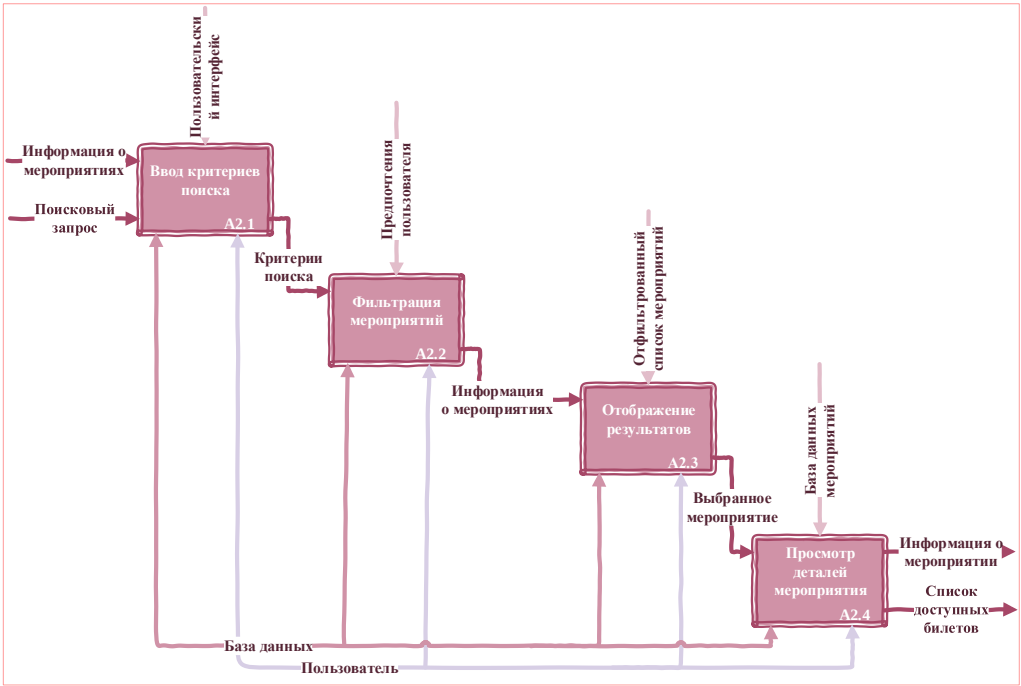


Рисунок 1.3 Декомпозиция процесса A2 " Поиск мероприятия "

Для упорядоченного отображения модели предметной области, используя принципы процессного управления, рассмотренные бизнес-процессы удобно представить в виде иерархической структуры – дерева бизнес-процессов. В этой структуре процессы распределяются по категориям: основные, поддерживающие и управленческие. Визуальное представление этой структуры представлено на рисунке 1.4.

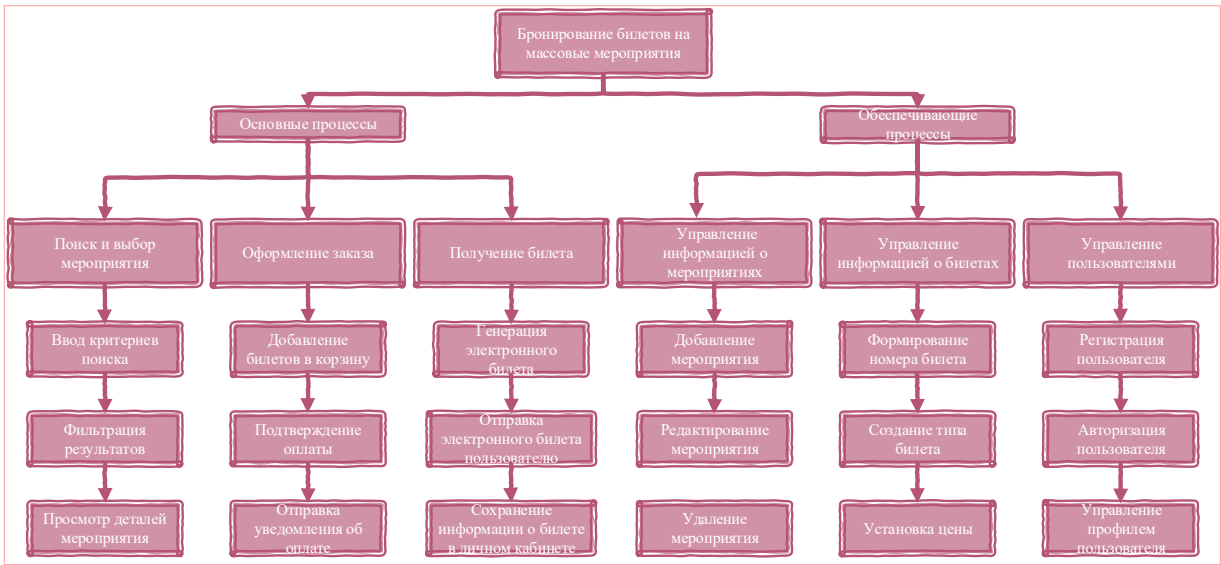


Рисунок 1.4 Дерево бизнес-процессов

Основной задачей классификации является облегчение работы по выделению процессов, снижение вероятности пропуска важных процессов, а также наглядное представление выделенных бизнес-процессов, разбитых на небольшие группы. Такой подход позволяет более эффективно проектировать и разрабатывать мобильное приложение для бронирования билетов, учитывая все ключевые аспекты предметной области.

1.2 Определение требований

Требования к разрабатываемому мобильному приложению представляют собой комплексное описание того, как приложение повлияет на бизнес заказчика (организатора мероприятий, билетного агрегатора и т.п.), и каким образом система должна вести себя при использовании конечными пользователями. Четкое определение требований необходимо для взаимопонимания между заказчиком и разработчиком, позволяя согласовать конечный результат разработки.

Процесс создания мобильного приложения начинается с выявления бизнес-требований, от которых зависят все последующие уровни спецификаций:

- Бизнес-требования: описывают, какую выгоду (финансовую, репутационную, операционную) бизнес планирует получить от внедрения мобильного приложения.
- Пользовательские требования: определяют, какие задачи и цели должны быть доступны пользователям приложения для достижения бизнес-целей.
- Функциональные требования: конкретизируют, какие функции и операции должно предоставлять приложение, чтобы пользователи могли выполнять свои задачи.

- Нефункциональные требования: описывают характеристики качества приложения, такие как производительность, безопасность, удобство использования, ограничения и особенности интерфейса.

Бизнес-требования (БТ) к мобильному приложению:

- Увеличение продаж билетов на массовые мероприятия.
- Расширение целевой аудитории за счет мобильных пользователей.
- Повышение узнаваемости бренда и лояльности клиентов.
- Оптимизация процесса бронирования и покупки билетов.
- Предоставление удобного канала для информирования о новых мероприятиях.
- Сбор и анализ данных о предпочтениях пользователей для улучшения маркетинговой стратегии.

Пользовательские требования описывают задачи, которые пользователи должны иметь возможность выполнять с помощью приложения. Эти требования можно представить в виде диаграммы, основанной на шагах бизнес-процесса бронирования билетов. (Рисунок 1.5)



Рисунок 1.5 Пользовательские требования.

Функциональные требования определяют, что именно должна делать система для удовлетворения потребностей пользователей. Взаимосвязь между пользовательскими и функциональными требованиями можно представить в виде диаграммы. (Рисунок 1.6)

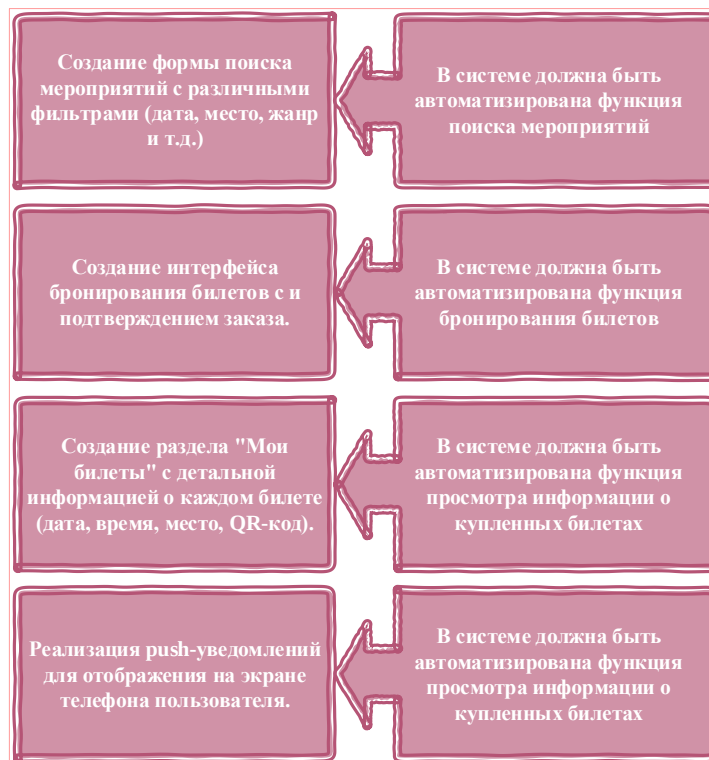


Рисунок 1.6 Взаимосвязь функциональных и пользовательских требований.

Описание зависимости между требованиями и функциями, реализующими эти требования, можно представить в виде таблицы (Таблица 1).

Таблица 1. Таблица функций, реализующих требования

№	Функциональное требование	Функции	Типовое проектное решение
1	Проектирование удобной авторизации и регистрации	Создание отдельных форм для входа существующих пользователей и регистрации новых, с минимальным количеством необходимых полей.	Разработка простого и интуитивно понятного дизайна форм, использование email-подтверждения для регистрации.
2	Реализация поиска мероприятий по различным параметрам	Реализация поиска с возможностью фильтрации по дате, месту, жанру, цене и другим критериям; Предоставление списка релевантных результатов с возможностью сортировки.	Применение алгоритмов полнотекстового поиска и индексации для быстрого поиска по названию

3	Предоставление подробной информации о мероприятии	Отображение названия, даты, времени, места проведения, описания, программы, списка участников (если есть), схемы.	Использование API карт для отображения местоположения
4	Бронирование билетов	Отображение стоимости билетов, расчет общей стоимости заказа.	Использование алгоритмов бронирования и резервирования
5	Хранение информации о купленных билетах в электронном виде	Создание личного кабинета пользователя, где хранится информация о всех купленных билетах, включая дату, время, место, номер билета, QR-код; Возможность просмотра билета в оффлайн-режиме.	Использование локальной базы данных для хранения информации о билетах, создание механизма синхронизации данных с сервером при наличии подключения к интернету.

Эта таблица представляет собой конкретный пример того, как функциональные требования связаны с функциями, которые необходимо разработать для реализации мобильного приложения.

Помимо функциональных требований, которые описывают, что должно делать приложение, существуют нефункциональные требования, определяющие как приложение должно это делать, а также его общие характеристики и ограничения. Они не связаны непосредственно с конкретными функциями, а описывают атрибуты качества, важные для пользователей и разработчиков.

Нефункциональные требования можно разделить на несколько категорий:

1. Производительность (скорость обработки запросов и время отклика),
2. Безопасность (защита данных от несанкционированного доступа),
3. Переносимость (совместимость с разными платформами и устройствами),
4. Юзабилити (удобство использования)
5. Масштабируемость (способность обрабатывать растущую нагрузку).

Таблица 2. Нефункциональные требования к системе

Категория	Требование	Значение
Время отклика	Допустимое время загрузки главной страницы, поиска мероприятий, страницы	< 3 секунды в обычном режиме

	мероприятия, страницы бронирования	
Пропускная способность	Максимальное количество одновременных пользователей	1000 активных пользователей
Обработка транзакций	Время обработки заказа на билеты	< 2 секунды в обычном режиме
Аутентификация и авторизация	Защита от несанкционированного доступа к данным пользователей	Использование стандартных безопасных протоколов и методов шифрования
Обработка ошибок	Грамотная обработка ошибок и исключений с понятными сообщениями для пользователя	Обязательное логирование всех ошибок
Поддерживаемые платформы	Android	Тестирование на устройствах
Юзабилити	Интуитивно понятный интерфейс. Легкость в навигации.	Проведение юзабилити-тестирования с целевой аудиторией

Нефункциональные требования важны как для пользователей, так и для разработчиков.

1.3 Анализ существующих решений на рынке программного обеспечения

На текущий момент на рынке мобильных приложений представлено множество решений для покупки билетов на массовые мероприятия. Эти решения позволяют пользователям находить интересующие их события, приобретать билеты в удобном формате и получать актуальную информацию о мероприятиях. Однако, существует необходимость в разработке приложения, которое учтет специфические требования и особенности целевой аудитории, а также предложит уникальный пользовательский опыт.

Для определения конкурентных преимуществ разрабатываемого приложения необходимо провести сравнительный анализ существующих программных продуктов. В качестве примеров для сравнения выбраны популярные платформы: Яндекс Билеты и Рупор. Эти приложения являются

известными игроками на рынке и предлагают широкий спектр услуг для бронирования билетов.

Таблица 3. Сравнительный анализ существующих программных продуктов

Критерий	Яндекс Билеты	Рупор	Разрабатываемое приложение
Функциональность ИС			
Поиск мероприятий	Широкий выбор мероприятий по жанру, дате, месту. Фильтры и рекомендации.	Мероприятия по городу, категории. Упор на массовые события.	Подробный и интуитивно понятный поиск, учитывающий персональные предпочтения пользователя.
Бронирование и покупка билетов	Удобный процесс бронирования, различные способы оплаты. Электронные билеты.	Бронирование и оплата билетов. Поддержка электронных билетов.	Простой и безопасный процесс покупки.
Личный кабинет пользователя	История покупок, избранные мероприятия, уведомления.	История покупок, избранные мероприятия.	История покупок, избранные мероприятия, уведомления.
Пользовательский интерфейс			
Удобство навигации	Интуитивно понятный интерфейс, простая навигация.	Удобный интерфейс, но может быть перегружен информацией.	Максимально простой и интуитивно понятный интерфейс, оптимизированный для мобильных устройств.
Визуальное оформление	Современный дизайн, приятная цветовая гамма.	Приятный дизайн, акцент на визуальном представлении событий.	Уникальный и запоминающийся дизайн, соответствующий фирменному стилю.
Технологические аспекты			
Платформы	iOS, Android, Web	iOS, Android, Web.	Планируется разработка под Android
Производительность	Высокая производительность, быстрая загрузка данных.	Высокая производительность.	Оптимизация для обеспечения высокой скорости работы и минимального потребления ресурсов.
Общая оценка			

Сильные стороны	Широкий охват мероприятий, удобная интеграция с другими сервисами Яндекса.	Упор на культурные события, удобный поиск.	Персонализация, уникальный пользовательский опыт, удобство использования.
Слабые стороны	Может быть перегружен информацией, отсутствие фокуса на конкретные типы мероприятий.	Ограниченный выбор мероприятий, не всегда удобная система фильтров.	Отсутствие большой группы разработчиков и бюджета.

Проведенный анализ показал, что на рынке существуют сильные решения с развитой функциональностью. Однако, существует возможность создать приложение, которое предложит уникальный пользовательский опыт, персонализированные рекомендации и максимально простой и интуитивно понятный интерфейс. Разрабатываемое приложение будет акцентировать внимание на потребностях пользователей и предложит удобные инструменты для поиска, бронирования и покупки билетов на интересующие мероприятия. Основной упор будет сделан на простоту использования и визуальную привлекательность, что позволит привлечь широкую аудиторию пользователей.

1.4. Составление проекта управлением разработкой ИС

Эффективное управление проектом разработки мобильного приложения для бронирования билетов на массовые мероприятия требует четкого определения требований к информационной системе (ИС). Это позволит структурировать процесс разработки, контролировать сроки и бюджет, а также обеспечить соответствие конечного продукта потребностям пользователей.

Проект управления разработкой ИС основывается на плане проекта, который может быть разработан с использованием инструментов управления проектами, таких как MS Project. План проекта включает в себя декомпозицию

работы на этапы (суммарные задачи) и подзадачи, образующие Структуру Декомпозиции Работ (СДР) или Иерархическую Структуру Работ (ИСР).

Иерархическая структура работ, показанная на рисунке 1.7, включает основные этапы разработки и внедрения приложения.

1	▲ "Разработка мобильного приложения для бронирования билетов на массовые мероприятия"	108 дней	Пт 20.12.24	Вт 20.05.25			1	1 608 800,00 Р
2	▸ Подготовка проекта	24 дней	Пт 20.12.24	Ср 22.01.25			1.1	268 000,00 Р
7	▸ Разработка программного решения	29 дней	Чт 23.01.25	Вт 04.03.25			1.2	580 000,00 Р
11	▸ Тестирование программного решения	23 дней	Ср 05.03.25	Пт 04.04.25			1.3	421 600,00 Р
15	▸ Внедрение системы	16 дней	Пн 07.04.25	Пн 28.04.25			1.4	153 600,00 Р
19	▸ Завершение проекта	16 дней	Вт 29.04.25	Вт 20.05.25			1.5	185 600,00 Р

Рисунок 1.7. Основные этапы разработки ИС.

В каждом этапе проекта есть перечень задач, описанных по типу, времени выполнения и ограничениям. Для точного планирования длительности следует учитывать их взаимосвязи (рис. 1.8).

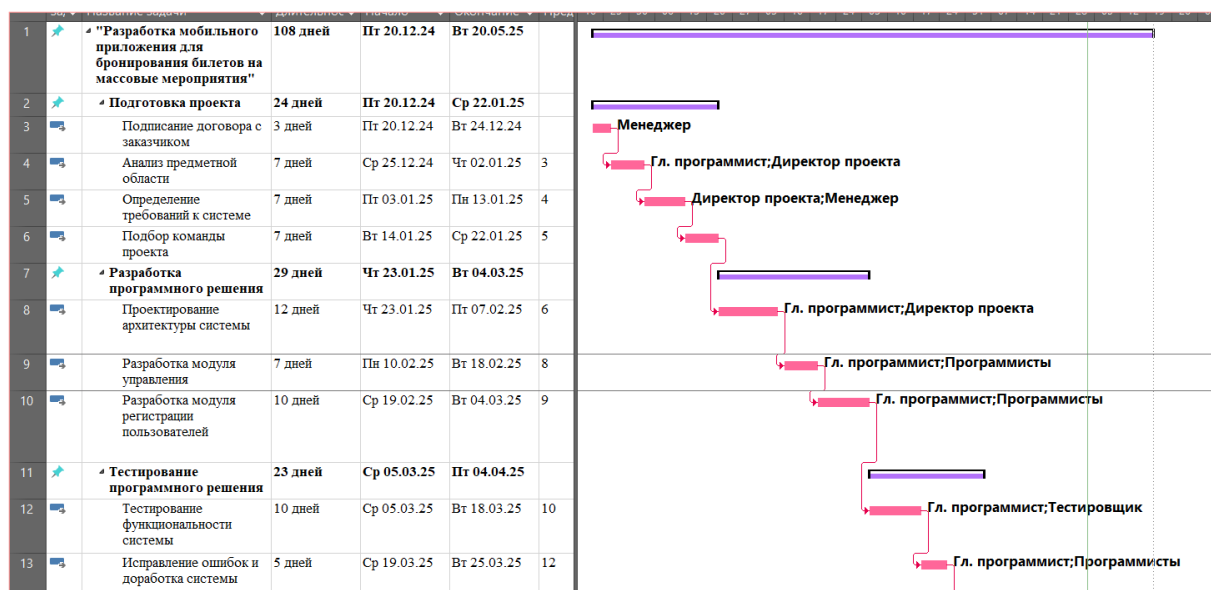


Рисунок 1.8. Иерархическая структура работ.

Выполнение работ требует создания проектной команды и выявления необходимых ресурсов. В реестр ресурсов записываются все нужные компоненты: материальные, трудовые и финансовые (Рис.1.9).

1	Директор проекта	Трудовой	Д	100%	1 000,00 Р/ч	0,00 Р/ч	0,00 Р	Пропорциональн	Стандартный
2	Гл. программист	Трудовой	Г	200%	1 500,00 Р/ч	0,00 Р/ч	0,00 Р	Пропорциональн	Стандартный
3	Менеджер	Трудовой	М	200%	900,00 Р/ч	0,00 Р/ч	0,00 Р	Пропорциональн	Стандартный
4	Тестировщик	Трудовой	Т	100%	1 000,00 Р/ч	0,00 Р/ч	0,00 Р	Пропорциональн	Стандартный
5	Программисты	Трудовой	П	100%	1 000,00 Р/ч	0,00 Р/ч	0,00 Р	Пропорциональн	Стандартный

Рисунок 1.9. Ресурсы проекта.

Для расчёта затрат на ресурсы нужно оценить стоимость самих ресурсов. Также следует определить стоимость материальных ресурсов и затрат. Важно обеспечить каждую задачу необходимыми ресурсами, поэтому на следующем этапе нужно назначить ресурсы на задачи.

Для каждой задачи в Microsoft Project рассчитываются общие затраты, которые включают фиксированные затраты и затраты на ресурсы. Структура общих затрат показана на рисунке 1.10.

за, ▾	Название задачи ▾	Длительность ▾	Начало ▾	Окончание ▾	Предш ▾	Названия ресурсов ▾	СДР ▾	Затраты
1	➤ "Разработка мобильного приложения для бронирования билетов на массовые мероприятия"	108 дней	Пт 20.12.24	Вт 20.05.25			1	1 608 800,00 Р
2	➤ Подготовка проекта	24 дней	Пт 20.12.24	Ср 22.01.25			1.1	268 000,00 Р
3	➤ Подписание договора с заказчиком	3 дней	Пт 20.12.24	Вт 24.12.24		Менеджер	1.1.1	21 600,00 Р
4	➤ Анализ предметной области	7 дней	Ср 25.12.24	Чт 02.01.25	3	Гл. программист;Директор	1.1.2	140 000,00 Р
5	➤ Определение требований к системе	7 дней	Пт 03.01.25	Пн 13.01.25	4	Директор проекта;Менеджер	1.1.3	106 400,00 Р
6	➤ Подбор команды проекта	7 дней	Вт 14.01.25	Ср 22.01.25	5		1.1.4	0,00 Р
7	➤ Разработка программного решения	29 дней	Чт 23.01.25	Вт 04.03.25			1.2	580 000,00 Р
8	➤ Проектирование архитектуры системы	12 дней	Чт 23.01.25	Пт 07.02.25	6	Гл. программист;Директор проекта	1.2.1	240 000,00 Р
9	➤ Разработка модуля управления	7 дней	Пн 10.02.25	Вт 18.02.25	8	Гл. программист;Программист	1.2.2	140 000,00 Р
10	➤ Разработка модуля регистрации пользователей	10 дней	Ср 19.02.25	Вт 04.03.25	9	Гл. программист;Программист	1.2.3	200 000,00 Р
11	➤ Тестирование программного решения	23 дней	Ср 05.03.25	Пт 04.04.25			1.3	421 600,00 Р
12	➤ Тестирование функциональности системы	10 дней	Ср 05.03.25	Вт 18.03.25	10	Гл. программист;Тестировщик	1.3.1	200 000,00 Р
13	➤ Исправление ошибок и доработка системы	5 дней	Ср 19.03.25	Вт 25.03.25	12	Гл. программист;Программист	1.3.2	100 000,00 Р

Рисунок 1.10. Назначение ресурсов и модель общих затрат.

В итоге длительность проекта составит – 108 дня, общая стоимость проекта – 1.608.800,00 рублей.

1.5. Разработка технического задания (Приложение 1)

1.6. Проектирование информационной системы

Для детальной проработки проекта используется язык UML (Unified Modeling Language) – универсальный язык визуального моделирования,

предназначенный для спецификации, визуализации, конструирования и документирования артефактов программного обеспечения, бизнес-процессов и других систем. UML позволяет представить систему в виде набора диаграмм.

Диаграмма прецедентов, изображённая на рисунке 1.11, является одним из основных инструментов UML для проектирования. Она описывает взаимодействие между акторами и прецедентами и позволяет представить функциональность системы на концептуальном уровне, определяя, кто может что делать с приложением.



Рисунок 1.11 Диаграмма прецедентов

Диаграмма классов UML используется для визуализации статической структуры системы, отображая классы, их атрибуты (данные), методы (операции) и взаимосвязи между ними. Это помогает в планировании базы данных и кода приложения.

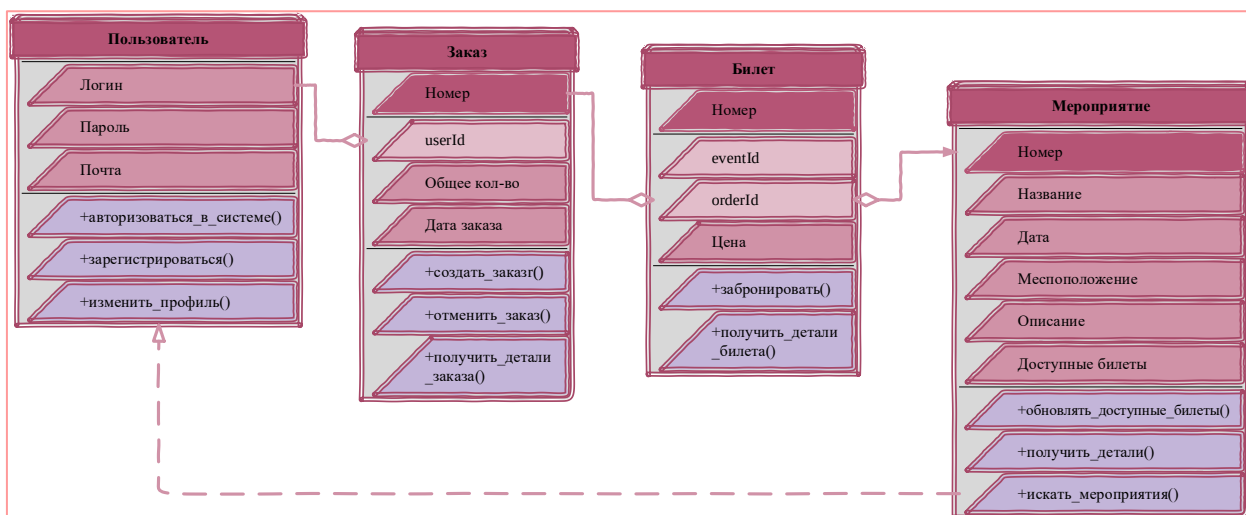


Рисунок 1.12 Диаграмма классов

Диаграмма состояний UML описывает различные состояния, в которых может находиться объект в течение своего жизненного цикла, а также переходы между этими состояниями, вызванные определенными событиями. Она визуализирует динамическое поведение системы, показывая, как сущности реагируют на внешние стимулы. (рис. 1.13)

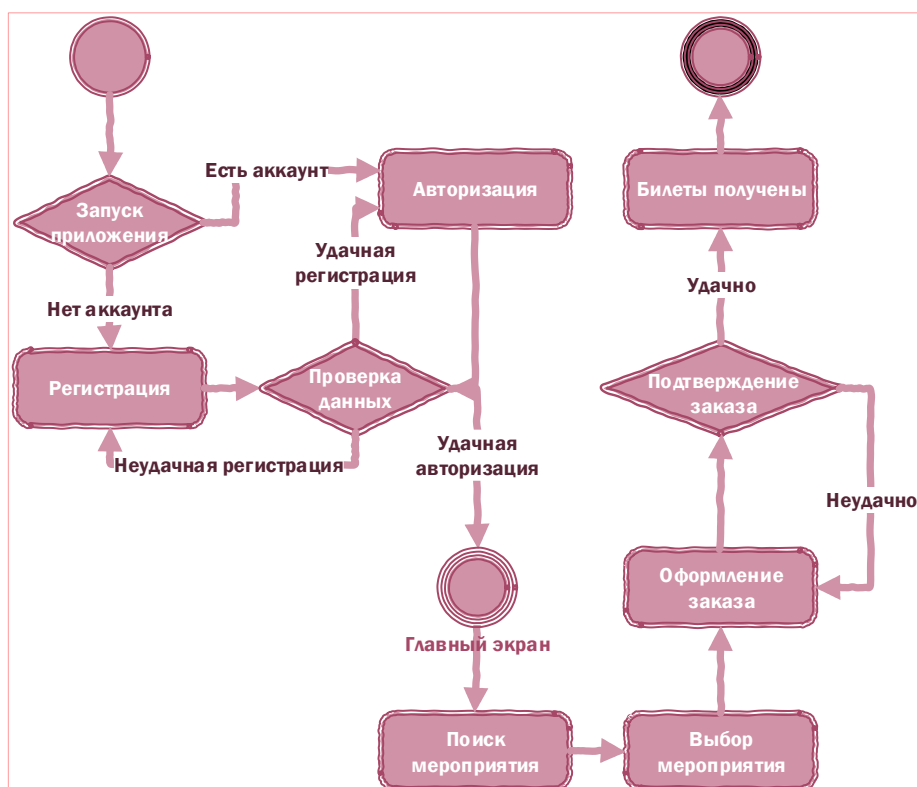


Рисунок 1.13 Диаграмма состояний

Диаграмма компонентов, представленная на рисунке 1.14, используется для наглядной демонстрации структуры компонентов системы и связей между

ними. С помощью диаграммы можно получить общее представление о компонентах системы.

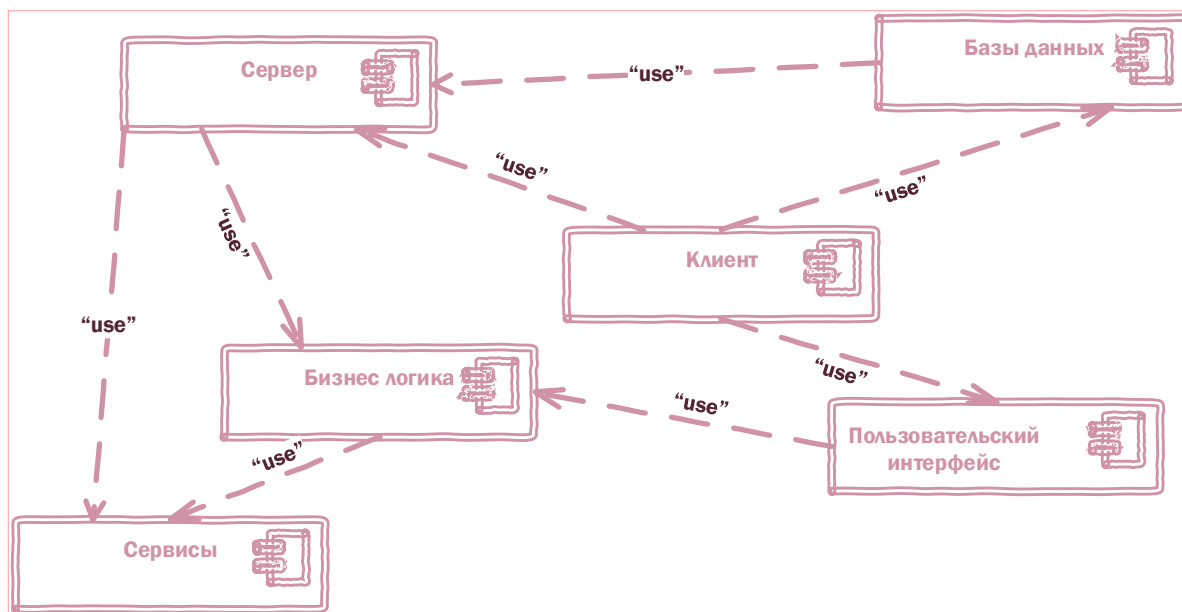


Рисунок 1.14 Диаграмма компонентов

После моделирования архитектуры программного обеспечения информационной системы с помощью диаграммы компонентов следует приступить к проектированию физической реализации системы. Первым шагом является разработка диаграммы развертывания. (рисунок 1.15)

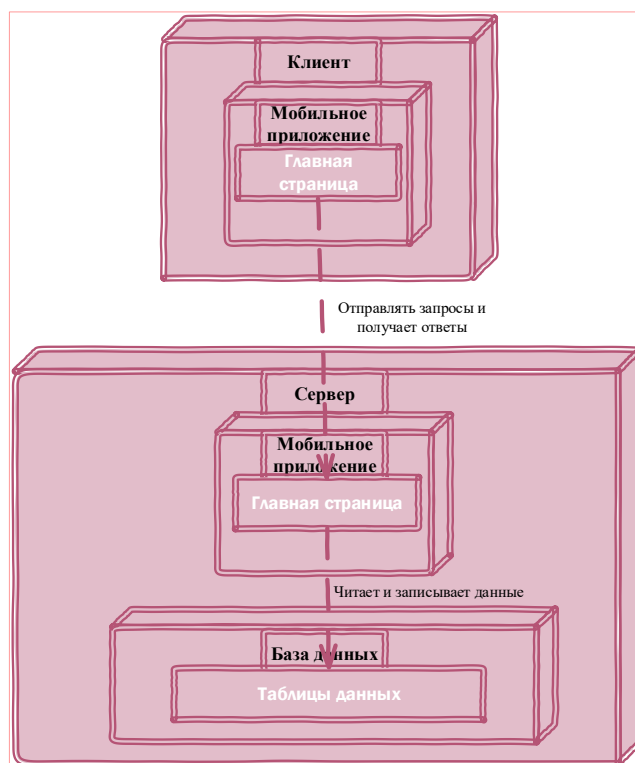


Рисунок 1.15. Диаграмма развёртывания

Архитектура информационной системы (ИС) – это концепция, которая определяет ее эталонную схему, структуру, реализуемые функции и взаимозависимость элементов. Проектирование архитектуры ИС производится путем сопоставления функциональных требований к ИС и ее элементам с функционалом, предоставляемым стандартными классами программного обеспечения и/или программных платформ.

Архитектуру информационной системы можно представить в виде следующих компонентов (Рис.2.7):

1. Слой отображения (Frontend) - клиентская часть с графическим пользовательским интерфейсом (GUI), которая служит в качестве терминала — средства отображения данных и передачи команд.
2. Слой бизнес-логики (Backend), где происходит обработка команд, полученных от клиента, и выполняются основные вычисления. В этом слое расположена система управления базами данных (СУБД) в качестве надстройки над базой данных (БД), которая позволяет обращаться к данным и управлять ими.
3. Слой доступа к данным, т.е. сама БД как хранилище данных в структурированном виде, что в конечном итоге на низком уровне сводится к файлам с записанными данными.

Опираясь на диаграмму развертывания, демонстрирующую сетевую организацию и распределение элементов системы по ее узлам, наряду с путями передачи данных между аппаратными компонентами, была задана архитектура информационной системы. Данная ИС будет использовать трехуровневую архитектуру «клиент-веб-сервер», где уровень бизнес-логики и уровень доступа к данным физически воплощаются на сервере, а уровень взаимодействия с пользователем размещается на клиентской стороне.

Поскольку компоненты системы располагаются на разных устройствах, им необходимо средство для взаимодействия – сеть передачи данных. Передача данных осуществляется в соответствии с сетевыми протоколами,

устанавливающими вид и структуру передаваемой информации. Схематическое изображение архитектуры ИС представлено на рисунке 1.16.

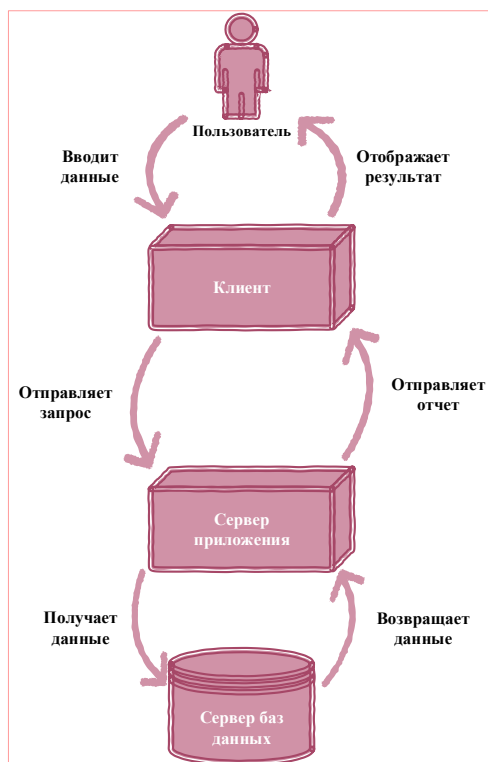


Рисунок 1.16. Схема трехуровневой клиент-серверной архитектуры.

Предложенная архитектура реализует трехуровневую клиент-серверную модель, в которой система разделена на три логических компонента: клиентский уровень, уровень бизнес-логики и уровень данных. Клиентский уровень отвечает за взаимодействие с пользователем, обеспечивая отображение информации и сбор пользовательского ввода через веб-интерфейсы, мобильные или десктопные приложения. Уровень бизнес-логики, выступая промежуточным звеном, обрабатывает основные бизнес-процессы приложения, управляет потоками данных и реализует ключевые функции системы. Уровень данных обеспечивает надежное хранение информации, поддерживая целостность и безопасность данных через реляционные или NoSQL базы данных и файловые хранилища. Такое разделение позволяет достичь высокой масштабируемости системы за счет возможности независимого масштабирования каждого уровня, упрощает поддержку и развитие приложения благодаря четкому разделению

ответственности, повышает безопасность за счет изоляции доступа к данным и улучшает отказоустойчивость системы. Трехуровневая архитектура особенно эффективна для сложных распределенных систем с высокими требованиями к производительности, масштабируемости и безопасности, предоставляя более гибкую и надежную альтернативу традиционной двухзвенной модели.

1.7. Проектирование базы данных

База данных будет состоять из нескольких взаимосвязанных сущностей, представленных в таблице 4.

Таблица 4. Список сущностей

№	Наименование сущности	Назначение
1	Пользователь (User)	Хранит информацию о зарегистрированных пользователях приложения (имя пользователя, пароль, mail, ID заказа)
2	Мероприятие (Event)	Хранит информацию о доступных мероприятиях (название, дата, местоположение, описание)
3	Билет (Ticket)	Хранит информацию о каждом билете, связанном с определенным мероприятием (ID мероприятия, цена)
4	Заказ (Order)	Хранит информацию о заказах, сделанных пользователями (ID билета, общая сумма, дата заказа)

Для каждой таблицы (сущности) необходимо определить атрибуты и их свойства. В таблице 5 приведены атрибуты таблиц «Пользователь (User)», «Мероприятие (Event)», «Билет (Ticket)», «Заказ (Order)».

Таблица 5. Атрибуты таблиц

Ключевое поле	Название атрибута	Назначение
Список атрибутов таблицы Пользователь (User)		
ПК (первичный ключ)	userId	Уникальный идентификатор пользователя
	username	Ник пользователя
	password	Пароль пользователя
	mail	Адрес электронной почты пользователя
БК (внешний ключ)	orderId	ID заказа пользователя
Список атрибутов таблицы Мероприятие (Event)		
ПК (первичный ключ)	eventId	Уникальный идентификатор мероприятия

	title	Название мероприятия
	date	Дата и время проведения мероприятия
	location	Место проведения мероприятия
	description	Описание мероприятия
Список атрибутов таблицы Билет (Ticket)		
ПК (первичный ключ)	ticketId	Уникальный идентификатор билета
БК (внешний ключ)	eventId	Идентификатор мероприятия, к которому относится билет
	price	Цена билета
	availableTickets	Количество билетов в наличии на данное мероприятие.
Список атрибутов таблицы Заказ (Order)		
ПК (первичный ключ)	orderId	Уникальный идентификатор заказа
БК (внешний ключ)	ticketId	Идентификатор билета, входящего в данный заказ
	totalAmount	Общая сумма заказа
	orderDate	Дата и время оформления заказа

Для поддержания ссылочной целостности необходимо определить в каждой сущности внешние ключи и связи между сущностями (таблица 6).

Таблица 6. Список связей

№	Название связи	Сущности, участвующие в связи	Назначение
1	1:M	Пользователь - Заказ	Один пользователь может оформить несколько заказов
2	1:M	Мероприятие - Билет	Одно мероприятие может иметь много билетов
3	1:M	Заказ - Билет	Один заказ может включать несколько билетов

Следующий этап включает в себя разработку инфологической модели данных, часто называемой ER-моделью (модель "сущность-связь"). Эта модель представляет собой графическую схему структуры базы данных, которая визуализирует основные объекты (сущности), их характеристики (атрибуты) и взаимоотношения между ними (рисунок 1.17).

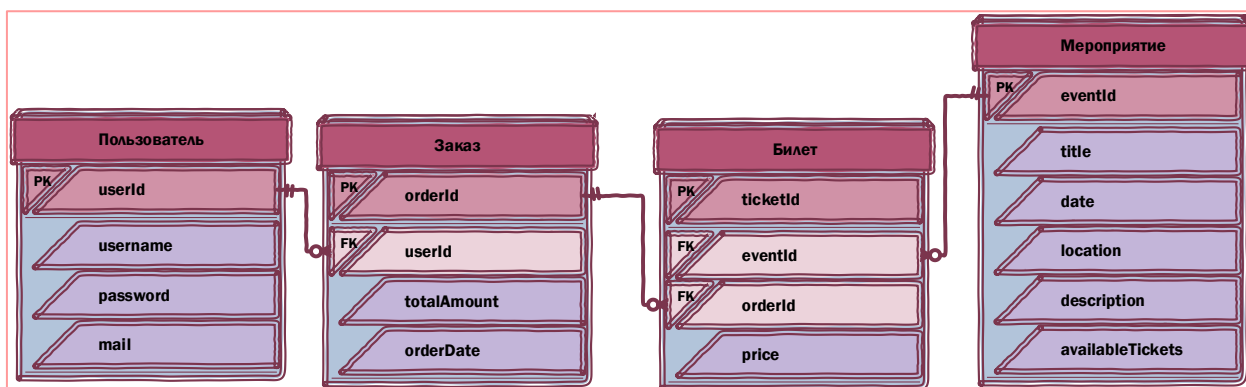


Рисунок 1.17 ER-модель.

В данном разделе было выполнено инфологическое проектирование базы данных для мобильного приложения бронирования билетов на массовые мероприятия, определены ключевые сущности, создан словарь базы данных, а также определены связи между сущностями, что обеспечивает основу для дальнейшей разработки приложения.

1.8. Проектирование интерфейса ИС

Пользовательские сценарии – список задач, которые может выполнять пользователь в рамках интерфейса. Все задачи расписываются по шагам, которые необходимо предпринять для решения задачи.

Действия, доступные пользователям:

1) Поиск мероприятий:

- Открыть приложение.
- Перейти на главный экран (список мероприятий).
- Ввести критерии поиска (город, дата, категория и т.д.).
- Просмотреть результаты поиска.
- Выбрать мероприятие.
- Просмотреть детали мероприятия (описание, дата, место, цена, доступные билеты).

2) Бронирование/Покупка билетов:

- На экране деталей мероприятия выбрать несколько билетов.

- Добавить билеты в корзину.
- Перейти в корзину.
- Проверить состав заказа.
- Авторизоваться (если еще не авторизован).
- Получить подтверждение заказа (на экране).

3) Просмотр профиля и истории заказов:

- Перейти в раздел "Профиль".
- Просмотреть личные данные.
- Изменить личные данные (при необходимости).
- Перейти в раздел "История заказов".
- Просмотреть список заказов.
- Просмотреть детали конкретного заказа (билеты, статус, сумма).

Действия, доступные администраторам.

1) Управление мероприятиями:

- Войти в систему с правами администратора.
- Перейти в раздел "Управление мероприятиями".
- Добавить новое мероприятие (заполнить форму с данными).
- Редактировать существующее мероприятие.
- Удалить мероприятие.

2) Управление пользователями:

- Перейти в раздел "Управление пользователями".
- Просмотреть список пользователей.
- Зарегистрировать нового пользователя (ввести данные).
- Редактировать данные пользователя.
- Удалить пользователя.

Проработанный ранее перечень шагов служит фундаментом для проектирования интерфейса. На его основе определяется количество экранов, их краткая характеристика и место в общей схеме. Итогом становится карта экранов (рис. 1.18).

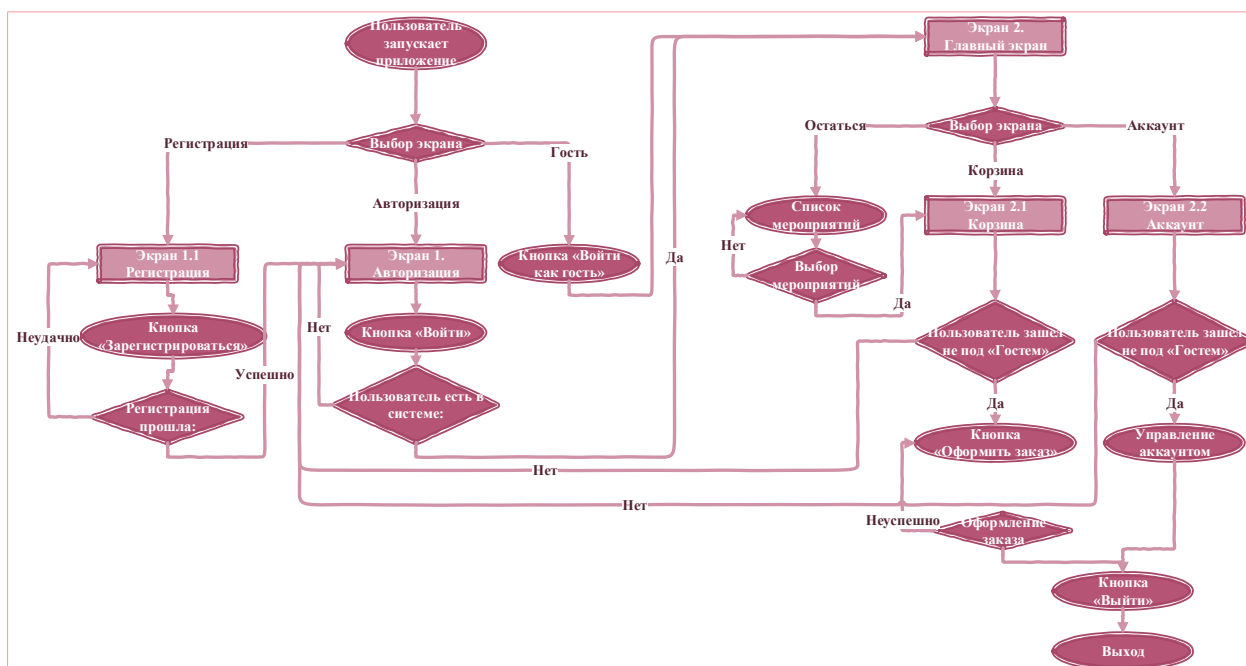


Рисунок 1.18 Карта экранов

На этапе проектирования интерфейса, после проведения исследований пользовательских предпочтений, была утверждена современная стилистика приложения. Для визуализации концепции разработаны тематические moodboard-коллажи.

Утверждённая стилевая система:

1. Цветовая палитра:

- Фиолетовый акцентный (#512BD4) - для интерактивных элементов
- Светлый фон (#FEF7FF) - основной фон интерфейса
- Тёмно-серый текст (#1C1B1F) - для контрастного отображения

2. Типографика:

- Основной шрифт: OpenSans Regular
- Акцентный шрифт: OpenSans Semibold

Пример цветов и шрифта, представлен на рисунке 1.19.

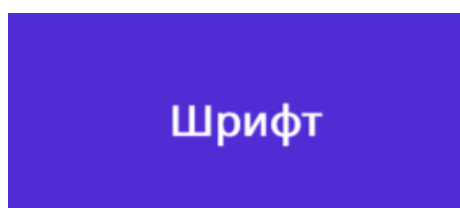


Рис. 1.19. Пример стилей

Экран авторизации и регистрации представлен на рисунке 1.20.

Рисунок 1.20. Экран авторизации и регистрации.

Главный экран показан на рисунке 1.21.

Рисунок 1.21. Главный экран

Экран корзина отображен на рисунке 1.22.



Рисунок 1.22. Экран корзина.

Экран настройки пользовательского аккаунта изображен на рисунке 1.23.

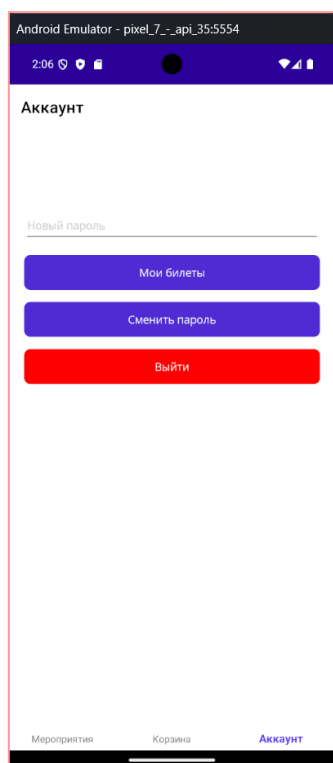


Рисунок 1.23. Экран Аккаунт.

Дальнейшее развитие проекта мобильного приложения включает в себя этап физического проектирования базы данных.

Вывод по теоретической части

Теоретическая часть дипломной работы включала анализ предметной области, разработку проекта управления разработкой ИС, проектирование информационной системы, базы данных, пользовательского интерфейса и разработку технического задания. На основе этих разработок будет проведена реализация физической модели базы данных, а также программной части информационной системы.

Глава 2. Практическая часть

2.1. Создание базы данных

Физическая модель базы данных реализуется в среде MS SQL Server. Таблица 7 содержит словарь метаданных для таблиц «Пользователь (User)», «Мероприятие (Event)», «Билет (Ticket)», «Заказ (Order)».

Таблица 7. Словарь Базы данных.

№	Название атрибута	Идентификатор	Тип	Не пусто	Ограничение
Таблица «Пользователь (User)»					
1	Идентификатор пользователя	userId	INT	Да	ПК (первичный ключ)
2	Ник пользователя	username	VARCHAR(8)	Да	Максимум – 8 символов
3	Пароль пользователя	password	VARCHAR(255)	Да	Максимум – 255 символов
4	Адрес электронной почты	email	VARCHAR(255)	Да	Максимум – 255 символов
Таблица «Мероприятие (Event)»					
1	Идентификатор мероприятия	eventId	INT	Да	ПК (первичный ключ)
2	Название мероприятия	title	VARCHAR(255)	Да	Максимум – 255 символов
3	Дата и время проведения	date	DATETIME	Да	
4	Место проведения	location	VARCHAR(255)	Да	Максимум – 255 символов
5	Описание мероприятия	description	TEXT	Нет	
6	Количество билетов в наличии	availableTickets	INT	Да	
Таблица Билет (Ticket)»					
1	Идентификатор билета	ticketId	INT	Да	ПК (первичный ключ)
2	Идентификатор мероприятия	eventId	INT	Да	ВК, ссылается на Event
3	Идентификатор заказа	orderId	INT	Да	ВК, ссылается на Order
4	Цена	price	DECIMAL(10,2)	Да	
Таблица «Заказ (Order)»					

1	Идентификатор заказа	orderId	INT	Да	ПК (первичный ключ)
2	Идентификатор пользователя	userId	INT	Да	ВК, ссылается на User
3	Общая сумма заказа	totalAmount	DECIMAL(10,2)	Да	
4	Дата и время оформления заказа	orderDate	DATETIME	Да	

Листинги SQL-запросов, используемых для создания таблиц с определением типов данных, ограничений целостности и ключей, представлены на рисунке 2.1.

```
-- Create User table
CREATE TABLE [User] (
    userId INT IDENTITY(1,1) PRIMARY KEY,
    username NVARCHAR(50) NOT NULL UNIQUE,
    password NVARCHAR(255) NOT NULL,
    mail NVARCHAR(100) NOT NULL UNIQUE,
    IsAdmin BIT DEFAULT 0 NOT NULL
);

-- Create Event table
CREATE TABLE [Event] (
    eventId INT IDENTITY(1,1) PRIMARY KEY,
    title NVARCHAR(100) NOT NULL,
    date DATETIME2 NOT NULL,
    location NVARCHAR(200) NOT NULL,
    description NVARCHAR(MAX),
    availableTickets INT NOT NULL CHECK (availableTickets >= 0)
);

-- Create Order table
CREATE TABLE [Order] (
    orderId INT IDENTITY(1,1) PRIMARY KEY,
    userId INT NOT NULL,
    totalAmount DECIMAL(10, 2) NOT NULL CHECK (totalAmount >= 0),
    orderDate DATETIME2 DEFAULT GETDATE() NOT NULL,
    CONSTRAINT FK_Order_User FOREIGN KEY (userId) REFERENCES [User](userId)
);

-- Create Ticket table
CREATE TABLE Ticket (
    ticketId INT IDENTITY(1,1) PRIMARY KEY,
    eventId INT NOT NULL,
    orderId INT NOT NULL,
    price DECIMAL(10, 2) NOT NULL CHECK (price >= 0),
    CONSTRAINT FK_Ticket_Event FOREIGN KEY (eventId) REFERENCES [Event](eventId),
    CONSTRAINT FK_Ticket_Order FOREIGN KEY (orderId) REFERENCES [Order](orderId)
);
```

Рисунок 2.1. Запрос создания таблиц БД.

На рисунке 2.2 представлена диаграмма базы данных, полученная в результате запроса, выполненного в SQL Server Management Studio.

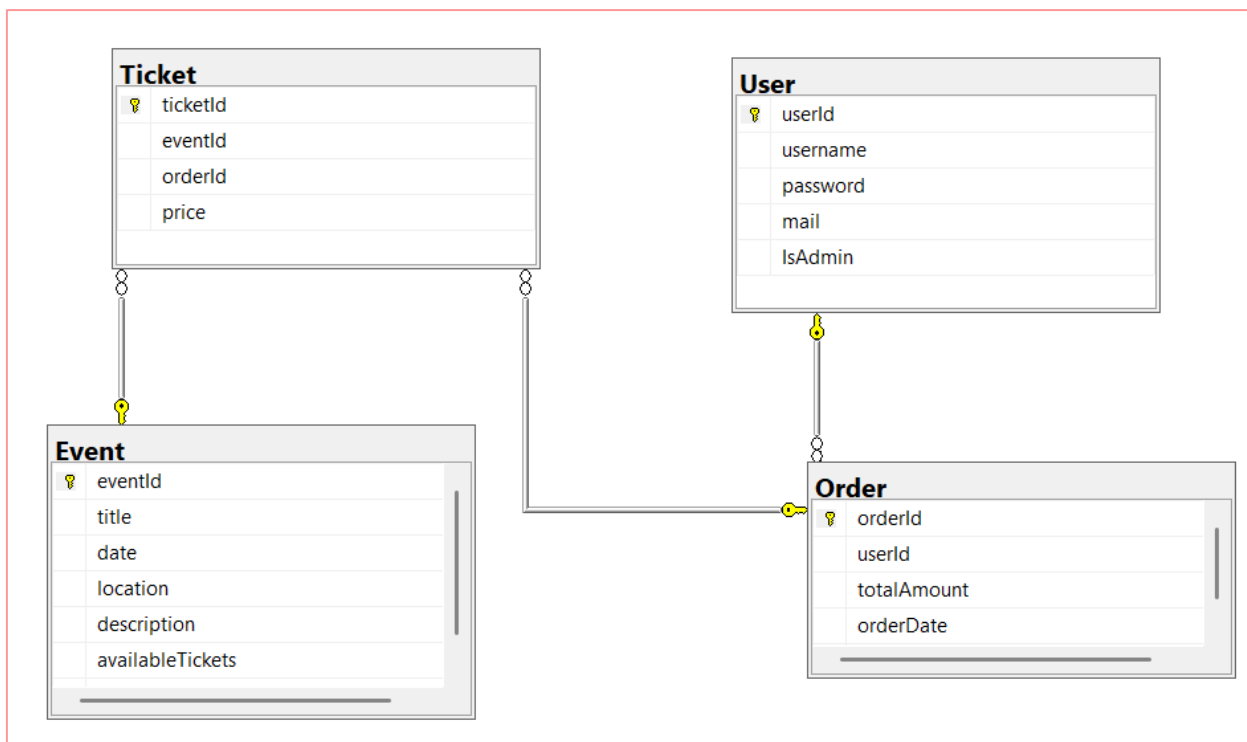


Рисунок 2.2. Диаграмма базы данных в приложении.

Взаимодействие с базой данных будет осуществляться с клиентского приложения с учетом роли пользователя.

2.2. Разработка клиентской части

Функциональная схема (рисунок 2.3) визуализирует структуру и взаимосвязи управляющих элементов разрабатываемой программной системы, диктуемые выбранной архитектурой.

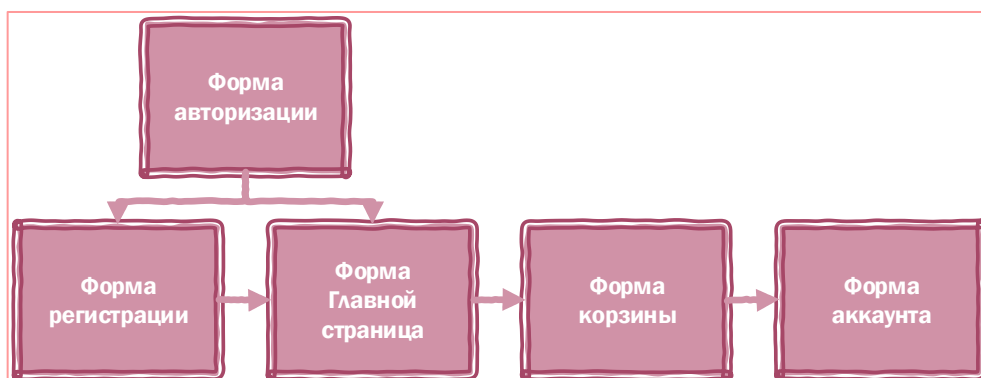


Рисунок 2.3. Функциональная схема

Вход в приложение для бронирования билетов осуществляется через экран выбора варианта входа (рисунок 2.4). Пользователь может зарегистрироваться, авторизоваться, либо войти как гость. При выборе авторизации, после нажатия кнопки «Войти» происходит проверка введенных учетных данных. Успешная авторизация открывает доступ к полному функционалу приложения (бронирование, управление аккаунтом, история покупок и т.д.), а именно, переводит пользователя на главный экран. Модуль проверяет наличие пользователя в БД и корректность введенных данных, при неудачной попытке выводит сообщение об ошибке. Ошибки подключения к БД обрабатываются и отображаются на всех экранах.

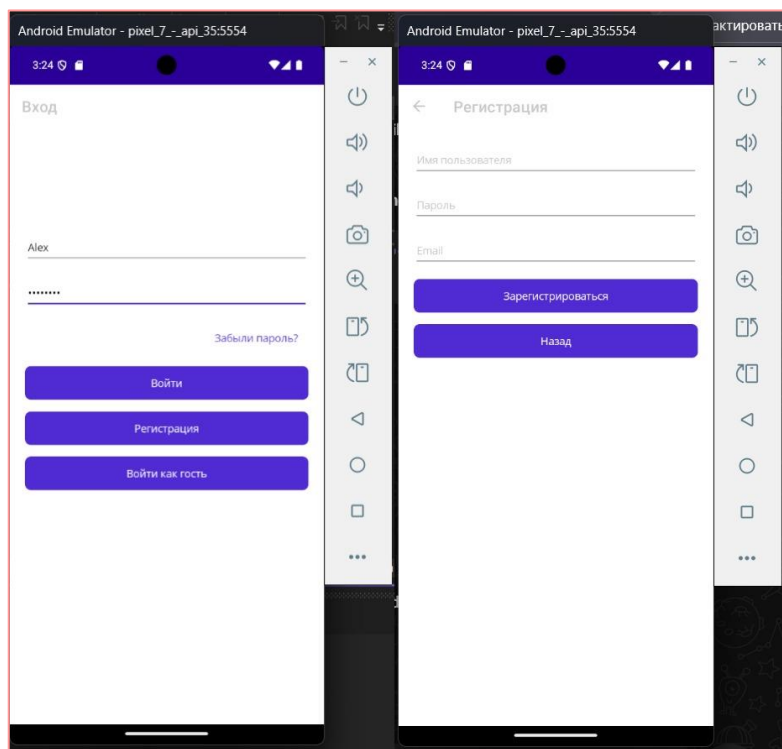


Рисунок 2.4. Реализация функции входа.

Основные экраны приложения: "Мероприятия" (рис. 2.5), "Корзина" (рис. 2.6), и "Аккаунт" (рис. 2.7) имеют схожую структуру, отличаясь набором отображаемых данных и кнопками навигации, зависящими от статуса пользователя (зарегистрирован или гость). Нажатие кнопок навигации позволяет перемещаться между этими экранами.

На экране "Мероприятия" отображается перечень доступных мероприятий, на экране "Корзина" – выбранные пользователем билеты, а на экране "Аккаунт" – информация о пользователе и его заказах.

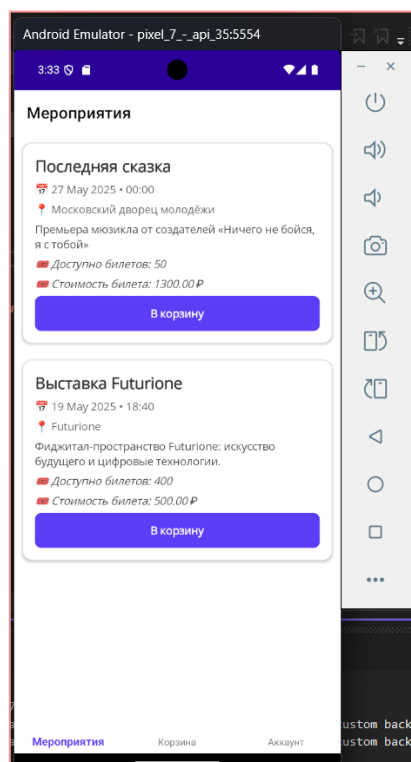


Рисунок 2.5. Экран Мероприятия.

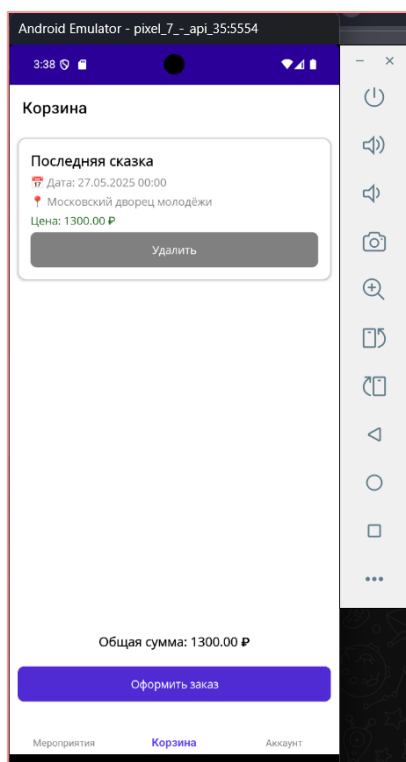


Рисунок 2.6. Экран «Корзина»

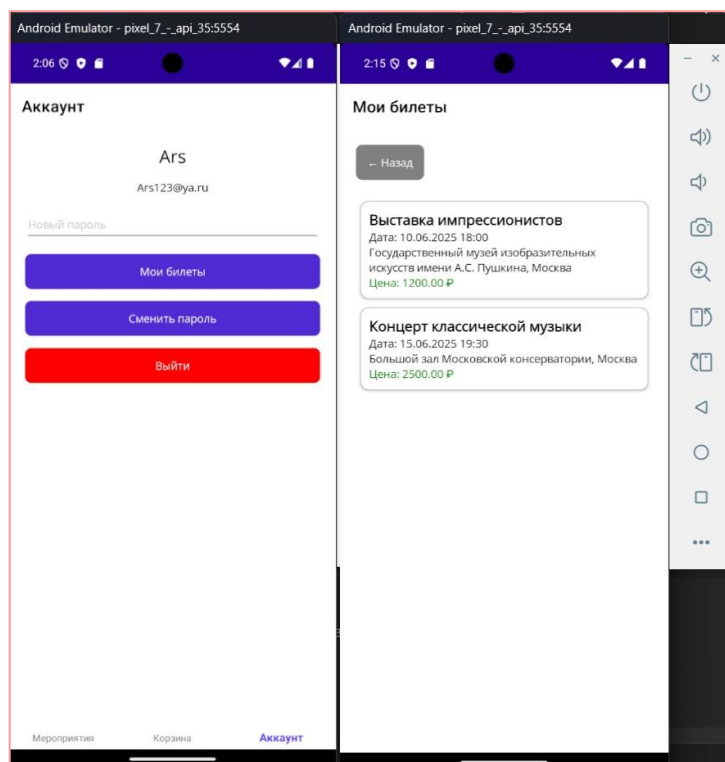


Рисунок 2.7. Экран «Аккаунт».

При открытии каждого из этих экранов происходит загрузка соответствующих данных из базы данных.

Для обеспечения надежности работы приложения во всех формах предусмотрена обработка возможных ошибок подключения к базе данных.

2.3 Тестирование информационной системы

В процессе разработки мобильного приложения для бронирования билетов важным этапом является его тестирование, направленное на поиск и устранение дефектов, несоответствий и сбоев в реализации функциональных требований. Функциональное тестирование, проводимое на первом этапе, необходимо для проверки соответствия функциональности требованиям, описанным в Таблице 8.

Таблица 8. Функциональное тестирование программы

№	Тестируемая функция	Действия	Ожидаемый результат	Фактический результат
1	Авторизация	Ввод неверных учетных данных	Отображение сообщения об ошибке авторизации	Соответствует ожидаемому

		Ввод корректных учетных данных	Переход на главный экран приложения (Список мероприятий)	Соответствует ожидаемому
2	Регистрация	Ввод некорректных данных	Отображение сообщений об ошибках валидации	Соответствует ожидаемому
		Ввод корректных данных	Создание новой учетной записи перенаправление на экран авторизации или главный экран	Соответствует ожидаемому
3	Просмотр информации о мероприятии	Выбор мероприятия из списка	Отображение подробной информации о мероприятии (название дата время место описание стоимость билетов)	Соответствует ожидаемому
4	Бронирование билетов	Выбор количества билетов и оформление заказа	Расчет общей стоимости заказа переход к экрану оплаты	Соответствует ожидаемому
5	Личный кабинет	Просмотр списка купленных билетов	Отображение информации о всех купленных билетах	Соответствует ожидаемому

Тестирование модуля "Список мероприятий" направлено на проверку корректности получения данных о мероприятиях. Для этого используется модульное тестирование с применением фреймворка MSTest. Результат одного из модульных тестов и его код представлен на рисунке 2.8.

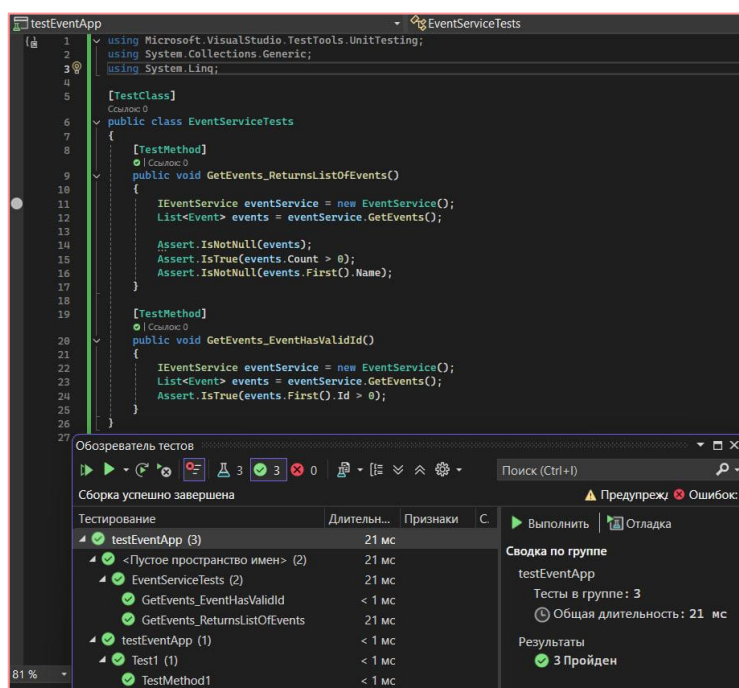


Рисунок 2.10. Тестирование MSTest.

Аналогичные тесты были выполнены для других ключевых функций приложения (регистрация, поиск мероприятий, бронирование билетов и т.д.). Модульное тестирование прошло успешно, значительных сбоев и ошибок не выявлено. Этап тестирования завершён.

2.4 Разработка руководства пользователя (Приложение №3)

Вывод по практической части

В рамках практической части дипломной работы было успешно разработано мобильное приложение для бронирования билетов на массовые мероприятия. Реализованы и протестированы ключевые функции: регистрация и авторизация пользователей, поиск мероприятий, просмотр информации о мероприятиях, бронирование билетов, личный кабинет пользователя. База данных, необходимая для функционирования приложения, создана и заполнена. Все выявленные в процессе тестирования ошибки были исправлены. Разработанная система полностью соответствует требованиям технического задания и готова к внедрению. Дизайн интерфейса выполнен в соответствии с концепцией, представленной в теоретической части работы.

Заключение

В ходе выполнения дипломной работы было выполнено проектирование и разработано мобильное приложение для бронирования билетов на массовые мероприятия, обеспечивающее удобство и эффективность процесса бронирования для пользователей.

В части проектирования большое внимание было уделено анализу предметной области поставленной задачи с целью выявления основных требований к разработке мобильного приложения. На основе выявленных требования было проведено детальное проектирование с использованием нотации UML. Вопросы хранения информации были решены при проектировании и реализации базы данных.

В процессе разработки реализованы следующие функции:

- Регистрация и авторизация пользователей.
- Просмотр подробной информации о мероприятиях.
- Бронирование билетов.
- Управление личным кабинетом пользователя.

Для обеспечения взаимодействия пользователей с приложением был разработан удобный пользовательский интерфейс. Работа приложения проверена с использованием функциональных и модульных тестов. В качестве завершающего этапа составлено руководство пользователя.

Цель и задачи дипломной работы выполнены в полном объеме.

Список использованных источников

1. Мартишин С.А., Симонов В.Л., Храпченко М.В. – Базы данных. Практическое применение СУБД SQL – и NoSQL-типа для применения проектирования информационных систем. Учебное пособие. Студентам ССУЗов – М.: Форум, 2023 г., – 447 с.
 2. Чистов Д.В., Мельников П.П., Золотарюк А.В., Ничепорук Н.Б. Проектирование информационных систем: учебник и практикум для среднего профессионального образования – М.: Юрайт, 2022 – 258 с.
 3. Кристин Марсикано, К. Стюарт, Билл Филлипс. – Android. Программирование для профессионалов. – Санкт-Петербург: Лань, 2018 – 704 с
 4. Ян Ф. Дарвин. – Android. Сборник рецептов. Задачи и решения для разработчиков приложений. – Диалектика, 2019 – 768 с
 5. Денис Колисниченко. – Программирование для Android. – М.: Юрайт, 2021 – 285 с
- Электронные ресурсы:
6. Руководство Microsoft: Создание схемы IDEF0. [Электронный ресурс]: <https://support.microsoft.com/ru-ru/topic/создание-схемы-idef0-ea7a9289-96e0-4df8-bb26-a62ea86417fc>
 7. Руководство Microsoft: Схемы UML в Visio. [Электронный ресурс]: <https://support.microsoft.com/ruru/office/%D1%81%D1%85%D0%B5%D0%BC%D1%8B-uml-%D0%B2-visioca4e3ae9-d413-4c94-8a7a-38dac30cbed6> , свободный.
 8. Руководство Microsoft: Создание схемы с нотацией базы данных Crow's Foot. [Электронный ресурс]: <https://support.microsoft.com/ru-ru/topic/создание-схемы-с-нотацией-базы-данных-вороньей-ноги-1ec22af9-3bd3-4354-b2b5-ed5752af6769> , свободный.

9. Статья: Создание проекта в MAUI. [Электронный ресурс]: learn.microsoft.com/ru-ru/dotnet/maui/get-started/first-app?view=net-maui-9.0, свободный.
10. Статья: Порядок создания технического задания для разработки информационной системы. [Электронный ресурс]: <https://habr.com/ru/articles/769460/>, свободный.
11. Статья: Как создать концепцию интерфейса. [Электронный ресурс]: <https://media.contented.ru/opyt/instrukcii/kak-sozdat-onczepczyuinterfejsa/>, свободный.

Техническое задание

На разработку и внедрение мобильного приложения для бронирования билетов на массовые мероприятия.

1. Общие сведения

1.1 Полное наименование системы и её условное обозначение

Полное наименование: Мобильное приложение для бронирования билетов на массовые мероприятия.

Краткое наименование: MBT (Mobile Booking Tickets).

1.2 Наименование организации-заказчика и описание деятельности

Заказчик: Акционерное общество «Яникс».

Место нахождения: Кирпичная улица, 33, Москва, 105187

Контактные данные: +7 (495) 134-12-34

1.3 Наименование организации-участников работ

Исполнитель: Рязанова Юлия Романовна

1.4 Определения, обозначения и сокращения

Система – мобильное приложение для бронирования билетов на массовые мероприятия.

Пользователь – физическое лицо, использующее приложение для бронирования билетов.

Администратор – пользователь с привилегированными правами для управления мероприятиями и пользователями.

Мероприятие – событие (концерт, спортивное соревнование и т.д.), на которое можно приобрести билеты.

Билет – электронный или физический документ, подтверждающий право посещения мероприятия.

2. Назначение и цели создания системы

2.1 Назначение системы

Система предназначена для автоматизации следующих процессов:

1. Регистрация и авторизация пользователей с распределением прав доступа (пользователь, администратор).
2. Поиск мероприятий с фильтрацией по дате, месту, жанру и другим параметрам.
3. Бронирование и покупка билетов с возможностью выбора количества и типа билетов.
4. Управление личным кабинетом, включая просмотр истории заказов и сохранённых мероприятий.
5. Управление мероприятиями и пользователями для администраторов.

2.2 Цели создания системы

1. Повышение удобства и скорости бронирования билетов для пользователей.
2. Увеличение продаж билетов за счёт удобного мобильного интерфейса.
3. Оптимизация работы организаторов мероприятий.
4. Обеспечение безопасности транзакций и защиты персональных данных.

3. Характеристики объекта автоматизации

3.1 Объект автоматизации

Автоматизации подлежат процессы:

1. Поиск и бронирование билетов.
2. Управление мероприятиями (добавление, редактирование, удаление).
3. Управление пользователями и их заказами.

4. Требования к системе

4.1 Требования к структуре и функционированию системы

1. Приложение должно быть разработано для платформы Android.
2. Использование реляционной базы данных (MS SQL Server, SQLite).
3. Обеспечение оффлайн-доступа к купленным билетам.

4.2 Требования к численности и квалификации персонала

1. Обучение администраторов работе с системой.

2. Базовые навыки работы с мобильными приложениями для пользователей.

4.3 Показатели назначения

1. Время отклика приложения: < 3 секунды.
2. Поддержка до 1000 одновременных пользователей.
3. Максимальное время обработки заказа: < 2 секунды.

4.4 Требования к надёжности

1. Круглосуточная доступность приложения.
2. Резервное копирование данных.
3. Восстановление после сбоев в течение 10 минут.

4.5 Требования к эргономике и технической эстетике

1. Интуитивно понятный интерфейс.
2. Русскоязычная локализация.
3. Современный дизайн с использованием фирменной цветовой палитры.

4.6 Требования к защите информации

1. Шифрование персональных данных.
2. Аутентификация пользователей.
3. Защита от несанкционированного доступа

5. Состав и содержание работ по созданию системы

Таблица 9. Состав работ по созданию ИС

Этапы работ	Результат	Дней
1. Подготовка проекта		
Подписание договора с заказчиком	Утверждённый контракт	3 дней
Анализ предметной области	Документ с анализом конкурентов и пользовательских сценариев	7 дней
Определение требований к системе	Техническое задание	7 дней
Подбор команды проекта	Сформированная команда	7 дней
2. Разработка программного решения		
Проектирование архитектуры системы	Диаграммы UML, ER-модель БД	12 дней
Разработка модуля управления	API для администраторов (добавление/редактирование мероприятий)	7 дней
Разработка модуля регистрации пользователей	Готовые экраны авторизации, мероприятий и аккаунта	10 дней
3. Тестирование программного решения		

Тестирование функциональности системы	Отчёт о тестах	10 дней
Исправление ошибок и доработка системы	Обновлённая версия приложения	5 дней
Подготовка документации по тестированию	Руководство по тестированию	8 дней
4. Внедрение системы		
Установка системы на сервере заказчика	Развёрнутое рабочее окружение	8 дней
Обучение пользователей работе с системой	Проведённые тренинги, FAQ-документ	3 дней
Сбор обратной связи от пользователей	Отчёт с предложениями по улучшению	5 дней
5. Завершение проекта		
Подготовка итогового отчета по проекту	Презентация для заказчика, отчёт о выполнении KPI	8 дней
Передача проекта заказчику	Акт сдачи-приёмки, архив с исходным кодом и документацией	8 дней
6. Ввод в эксплуатацию		
Эксплуатация системы	Полноценная работа Системы	Срок не определён

6. Порядок контроля и приемки системы

Для проверки соответствия мобильного приложения "МВТ" установленным функциональным и техническим требованиям предусматривается поэтапная процедура контроля качества и приемки. Процесс верификации системы включает три последовательных вида испытаний: тестовые испытания, опытную эксплуатацию и приемочные испытания.

Тестовые испытания проводятся разработчиками на этапе с 05 марта по 18 марта 2025 года. В ходе данного этапа осуществляется комплексная проверка работоспособности всех модулей приложения, включая функционал поиска мероприятий, бронирования билетов и проведения платежей. Особое внимание уделяется соответствию пользовательского интерфейса утвержденным дизайн-макетам, а также проводится нагрузочное тестирование для подтверждения стабильной работы системы при одновременном подключении до 1000 пользователей. Результаты тестовых испытаний фиксируются в протоколе, содержащем перечень выявленных дефектов и рекомендации по их устранению.

Опытная эксплуатация системы запланирована на период с 22 по 28 апреля 2025 года. На данном этапе к тестированию привлекаются внутренние специалисты компании-заказчика, которые проводят оценку приложения в условиях, максимально приближенных к реальной эксплуатации. Основными задачами опытной эксплуатации являются проверка удобства пользовательского интерфейса, анализ соответствия фактического поведения системы ожиданиям конечных пользователей и сбор предложений по улучшению функциональности. По завершении данного этапа составляется акт о завершении опытной эксплуатации, содержащий перечень выполненных доработок.

Приемочные испытания являются заключительным этапом проверки и проводятся с 09 по 20 мая 2025 года. Критерием успешного прохождения данного этапа является устранение всех критических замечаний, выявленных в ходе предыдущих испытаний, а также полное соответствие системы требованиям технического задания и стандартам ГОСТ 34.201-89. Приемка осуществляется специальной комиссией, в состав которой входят представители заказчика и разработчика. По результатам испытаний подписывается акт приемки в промышленную эксплуатацию и, при необходимости, оформляется сертификат соответствия.

7. Требования к документированию

Для обеспечения эффективной эксплуатации и сопровождения системы необходимо разработать полный комплект документации, соответствующий требованиям ГОСТ 34.201-89, отраслевым НТД Заказчика, ЕСКД и ЕСПД. В состав сопроводительной документации должны входить:

1. Описание проектного решения, содержащее информацию о функциональности и настройках системы.
2. Учебные материалы для проведения курсов по внедрению системы.
3. Ролевая модель, описывающая права доступа различных групп пользователей.

4. Руководство администратора, содержащее детальные инструкции по установке, сопровождению и восстановлению системы.
5. Руководство пользователя, предназначенное для конечных пользователей системы.

ПРИЛОЖЕНИЕ №2

Листинг страницы авторизации:

```
using EventApp.Services;
using EventApp.ViewModels.Shared;
using EventApp.Views.Auth;
using EventApp.Views.User;
using System.Threading.Tasks;
using System.Windows.Input;

namespace EventApp.ViewModels.Auth
{
    public class LoginViewModel : BaseViewModel
    {
        private readonly AuthService _authService;
        private string _username;
        private string _password;
        private bool _isBusy;

        public string Username
        {
            get => _username;
            set => SetProperty(ref _username, value);
        }

        public string Password
        {
            get => _password;
            set => SetProperty(ref _password, value);
        }

        public bool IsBusy
        {
            get => _isBusy;
            set => SetProperty(ref _isBusy, value);
        }

        public ICommand LoginCommand { get; }
        public ICommand RegisterCommand { get; }
        public ICommand GuestCommand { get; }
        public ICommand ForgotPasswordCommand { get; }

        public LoginViewModel(AuthService authService)
        {
            _authService = authService;
            LoginCommand = new Command(async () => await Login());
            RegisterCommand = new Command(async () => await Register());
            GuestCommand = new Command(async () => await GuestLogin());
            ForgotPasswordCommand = new Command(async () => await ForgotPassword());
        }
    }
}
```

```

private async Task Login()
{
    if (IsBusy) return;
    IsBusy = true;

    var success = await _authService.Login(Uusername, Password);
    if (success)
    {
        Application.Current.MainPage = new AppShell();
    }
    else
    {
        await Application.Current.MainPage.DisplayAlert("Ошибка", "Неверный логин или пароль", "OK");
    }

    IsBusy = false;
}

private async Task GuestLogin()
{
    Application.Current.MainPage = new AppShell();
    await Shell.Current.GoToAsync("//EventsPage");
}

private async Task Register()
{
    var vm = App.ServiceProvider.GetRequiredService<RegisterViewModel>();
    var page = new RegisterPage(vm);

    await Application.Current.MainPage.Navigation.PushAsync(page);
}

private async Task ForgotPassword()
{
    var vm = App.ServiceProvider.GetRequiredService<PasswordResetViewModel>();
    var page = new PasswordResetPage(vm);

    await Application.Current.MainPage.Navigation.PushAsync(page);
}
}
}

```

Листинг страницы регистрации:

```

using EventApp.Models;
using EventApp.Services;
using EventApp.ViewModels.Shared;
using System.Threading.Tasks;
using System.Windows.Input;

namespace EventApp.ViewModels.Auth
{
    public class RegisterViewModel : BaseViewModel
    {
        private readonly AuthService _authService;
        private string _username;
        private string _password;
        private string _email;
        private bool _isBusy;

        public string Username
        {
            get => _username;

```

```

        set => SetProperty(ref _username, value);
    }

    public string Password
    {
        get => _password;
        set => SetProperty(ref _password, value);
    }

    public string Email
    {
        get => _email;
        set => SetProperty(ref _email, value);
    }

    public bool IsBusy
    {
        get => _isBusy;
        set => SetProperty(ref _isBusy, value);
    }

    public ICommand RegisterCommand { get; }
    public ICommand BackCommand { get; }

    public RegisterViewModel(AuthService authService)
    {
        _authService = authService;
        RegisterCommand = new Command(async () => await Register());
        BackCommand = new Command(async () => await Back());
    }

    private async Task Register()
    {
        if (IsBusy) return;
        IsBusy = true;

        var user = new User
        {
            Username = Username,
            Password = Password,
            Mail = Email,
            IsAdmin = false
        };

        var success = await _authService.Register(user);
        if (success)
        {
            await Application.Current.MainPage.DisplayAlert("Успех", "Регистрация прошла успешно", "OK");
            await Shell.Current.GoToAsync("//Login");
        }
        else
        {
            await Application.Current.MainPage.DisplayAlert("Ошибка", "Не удалось зарегистрироваться", "OK");
        }

        IsBusy = false;
    }

    private async Task Back()
    {
        await Shell.Current.GoToAsync("//Login");
    }
}

```

Листинг страницы мероприятий (главная страница):

```
using EventApp.Models;
using EventApp.Services;
using EventApp.ViewModels.Shared;
using System.Collections.Generic;
using System.Threading.Tasks;
using System.Windows.Input;

namespace EventApp.ViewModels.UserViewModels
{
    public class EventsViewModel : BaseViewModel
    {
        private readonly IDatabaseService _databaseService;
        private readonly AuthService _authService;

        private List<Event> _events;
        private bool _isBusy;

        public List<Event> Events
        {
            get => _events;
            set => SetProperty(ref _events, value);
        }

        public bool IsBusy
        {
            get => _isBusy;
            set => SetProperty(ref _isBusy, value);
        }

        public ICommand LoadEventsCommand { get; }
        public ICommand AddToCartCommand { get; }

        public EventsViewModel(IDatabaseService databaseService, AuthService authService)
        {
            _databaseService = databaseService;
            _authService = authService;

            LoadEventsCommand = new Command(async () => await LoadEvents());
            AddToCartCommand = new Command<Event>(async (e) => await AddToCart(e));
        }

        private async Task LoadEvents()
        {
            if (IsBusy) return;
            IsBusy = true;

            Events = await _databaseService.GetAllEvents();
            OnPropertyChanged(nameof(Events));

            IsBusy = false;
        }

        private async Task AddToCart(Event @event)
        {
            if (IsBusy) return;
            IsBusy = true;
            if (_authService.CurrentUser == null)
            {
                await Application.Current.MainPage.DisplayAlert("Ошибка", "Пожалуйста, авторизуйтесь, чтобы добавить в корзину", "OK");
                await Shell.Current.GoToAsync("//LoginPage");
            }
        }
    }
}
```

```

        IsBusy = false;
        return;
    }
    var order = new Order
    {
        UserId = _authService.CurrentUser.UserId,
        EventId = @event.EventId,
        TotalAmount = @event.Price,
        OrderDate = DateTime.Now
    };

    var result = await _databaseService.AddOrderToCart(order);

    if (result)
    {
        await Application.Current.MainPage.DisplayAlert("Успех", "Мероприятие добавлено в корзину", "ОК");
    }
    else
    {
        await Application.Current.MainPage.DisplayAlert("Ошибка", "Не удалось добавить в корзину", "ОК");
    }

    IsBusy = false;
}
}

```

Листинг страницы корзина:

```

using EventApp.DTO;
using EventApp.Models;
using EventApp.Services;
using EventApp.ViewModels.Shared;
using System.Collections.Generic;
using System.Threading.Tasks;
using System.Windows.Input;

namespace EventApp.ViewModels.UserViewModels
{
    public class CartViewModel : BaseViewModel
    {
        private readonly IDatabaseService _databaseService;
        private readonly AuthService _authService;
        private List<CartItemDisplay> _cartItems;
        private bool _isBusy;

        public List<CartItemDisplay> CartItems
        {
            get => _cartItems;
            set => SetProperty(ref _cartItems, value);
        }

        public bool IsBusy
        {
            get => _isBusy;
            set => SetProperty(ref _isBusy, value);
        }

        private decimal _totalPrice;
        public decimal TotalPrice
        {

```

```

        get => _totalPrice;
        set => SetProperty(ref _totalPrice, value);
    }

    public ICommand LoadCartCommand { get; }
    public ICommand CheckoutCommand { get; }
    public ICommand RemoveFromCartCommand { get; }

    public CartViewModel(IDatabaseService databaseService, AuthService authService)
    {
        _databaseService = databaseService;
        _authService = authService;
        LoadCartCommand = new Command(async () => await LoadCart());
        CheckoutCommand = new Command(async () => await Checkout());
        RemoveFromCartCommand = new Command<CartItemDisplay>(async (item) => await RemoveFromCart(item));
    }

    private async Task LoadCart()
    {
        if (IsBusy) return;
        IsBusy = true;

        if (_authService.CurrentUser == null)
        {
            await Application.Current.MainPage.DisplayAlert("Ошибка", "Вы не авторизованы", "OK");
            await Shell.Current.GoToAsync("//LoginPage");
            IsBusy = false;
            return;
        }

        var orders = await _databaseService.GetOrdersForUser(_authService.CurrentUser.UserId);
        var result = new List<CartItemDisplay>();

        decimal total = 0;

        foreach (var order in orders)
        {
            var ev = await _databaseService.GetEventById(order.EventId);
            result.Add(new CartItemDisplay
            {
                OrderId = order.OrderId,
                EventTitle = ev.Title,
                EventDate = ev.Date,
                Location = ev.Location,
                TotalAmount = order.TotalAmount
            });

            total += order.TotalAmount;
        }

        CartItems = result;
        TotalPrice = total;
        IsBusy = false;
    }

    private async Task Checkout()
    {
        if (IsBusy) return;
        IsBusy = true;
        await Application.Current.MainPage.DisplayAlert("Успех", "Заказ оформлен", "OK");

        IsBusy = false;
    }

```

```

private async Task RemoveFromCart(CartItemDisplay item)
{
    if (IsBusy) return;
    IsBusy = true;

    try
    {
        var resultOrder = await _databaseService.DeleteOrderId(item.OrderId);
        if (!resultOrder)
        {
            await Application.Current.MainPage.DisplayAlert("Ошибка", "Не удалось удалить заказ из корзины",
"OK");
            return;
        }
        CartItems.Remove(item);
        OnPropertyChanged(nameof(CartItems));

        TotalPrice -= item.TotalAmount;
        OnPropertyChanged(nameof(TotalPrice));

        await Application.Current.MainPage.DisplayAlert("Успех", "Товар удален из корзины", "OK");
    }
    catch (Exception ex)
    {
        await Application.Current.MainPage.DisplayAlert("Ошибка", ex.Message, "OK");
    }
    finally
    {
        IsBusy = false;
    }
}
}

```

Листинг страницы аккаунт:

```

using EventApp.Models;
using EventApp.Services;
using EventApp.ViewModels.Shared;
using EventApp.Views.User;
using System.Threading.Tasks;
using System.Windows.Input;

namespace EventApp.ViewModels.UserViewModels
{
    public class AccountViewModel : BaseViewModel
    {
        private readonly AuthService _authService;
        private readonly IDatabaseService _databaseService;
        private Models.User _currentUser;
        private string _newPassword;
        private bool _isBusy;

        public Models.User CurrentUser
        {
            get => _currentUser;
            set => SetProperty(ref _currentUser, value);
        }

        public string NewPassword
        {
            get => _newPassword;

```

```

        set => SetProperty(ref _newPassword, value);
    }

    public bool IsBusy
    {
        get => _isBusy;
        set => SetProperty(ref _isBusy, value);
    }

    public ICommand LoadUserCommand { get; }
    public ICommand UpdatePasswordCommand { get; }
    public ICommand LogoutCommand { get; }

    public AccountViewModel(AuthService authService, IDatabaseService databaseService)
    {
        _authService = authService;
        _databaseService = databaseService;
        LoadUserCommand = new Command(async () => await LoadUser());
        UpdatePasswordCommand = new Command(async () => await UpdatePassword());
        LogoutCommand = new Command(async () => await Logout());
    }

    private async Task LoadUser()
    {
        if (IsBusy) return;
        IsBusy = true;

        CurrentUser = _authService.CurrentUser;

        IsBusy = false;
    }

    private async Task UpdatePassword()
    {
        if (IsBusy) return;
        IsBusy = true;

        var success = await _databaseService.UpdatePassword(CurrentUser.UserId, NewPassword);
        if (success)
        {
            await Application.Current.MainPage.DisplayAlert("Успех", "Пароль изменен", "OK");
            NewPassword = string.Empty;
        }
        else
        {
            await Application.Current.MainPage.DisplayAlert("Ошибка", "Не удалось изменить пароль", "OK");
        }

        IsBusy = false;
    }

    private async Task Logout()
    {
        _authService.Logout();
        Application.Current.MainPage = new AppShell();
        await Shell.Current.GoToAsync("//Login");
    }
}

```


Руководство пользователя

1. Запуск приложения

Нажмите на иконку приложения MBT (Mobile Booking Tickets) на вашем устройстве. После запуска откроется экран авторизации.

2. Авторизация и регистрация

2.1 Вход в аккаунт

Введите логин (username) и пароль в соответствующие поля. Нажмите кнопку «Войти», если данные введены неверно, появится сообщение об ошибке. Проверьте корректность ввода и попробуйте снова.

2.2 Регистрация нового пользователя

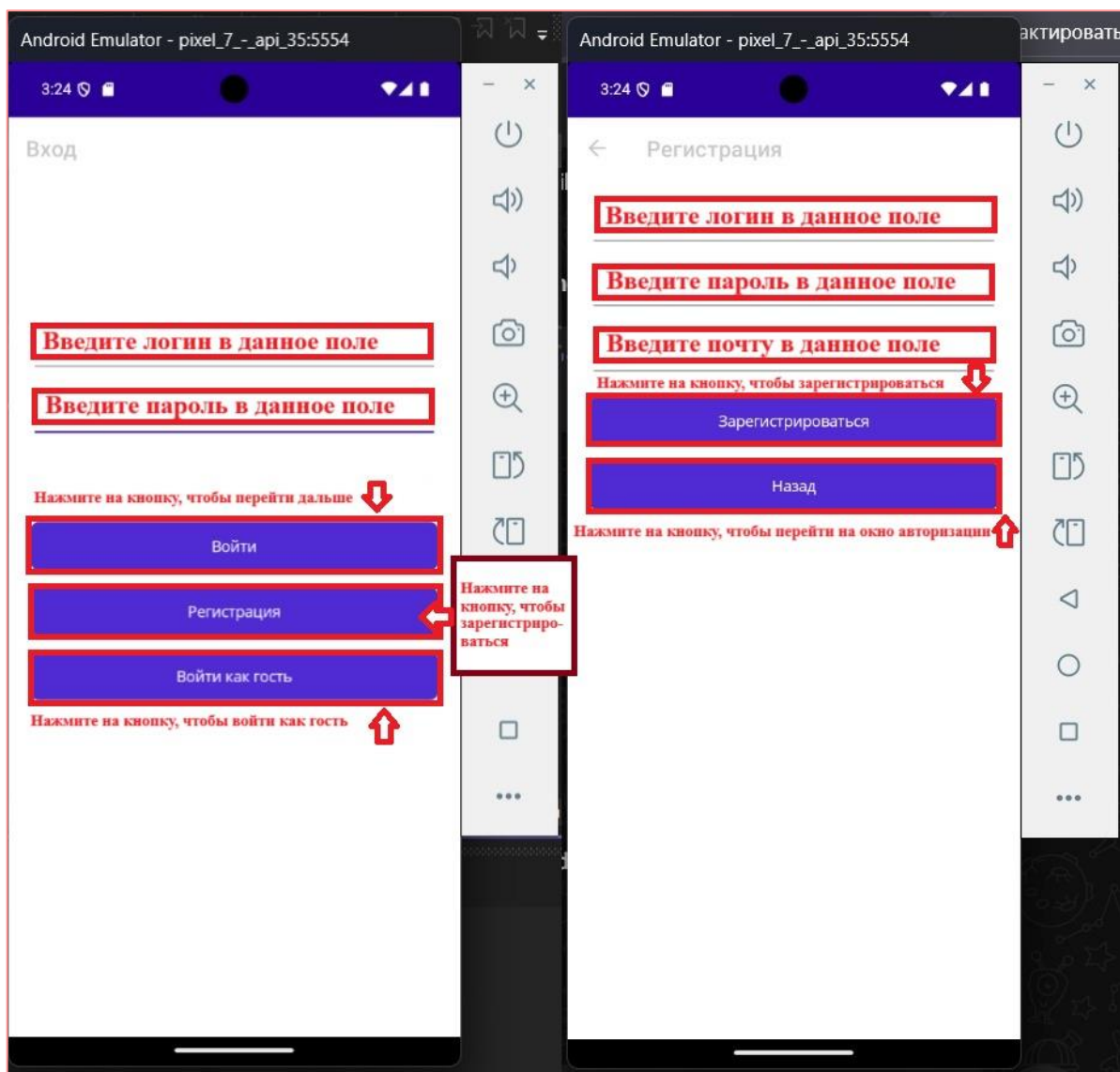
Нажмите кнопку «Зарегистрироваться». Заполните поля:

- a. Email
- b. Логин (никнейм)
- c. Пароль (не менее 8 символов)

Нажмите «Зарегистрироваться». После успешной регистрации вы автоматически перейдете на страницу авторизации.

2.3 Вход без регистрации (гостевой режим)

Нажмите «Войти как гость». В этом режиме доступен просмотр мероприятий, но бронирование билетов и управление аккаунтом потребует авторизации.



3. Главный экран (Мероприятия)

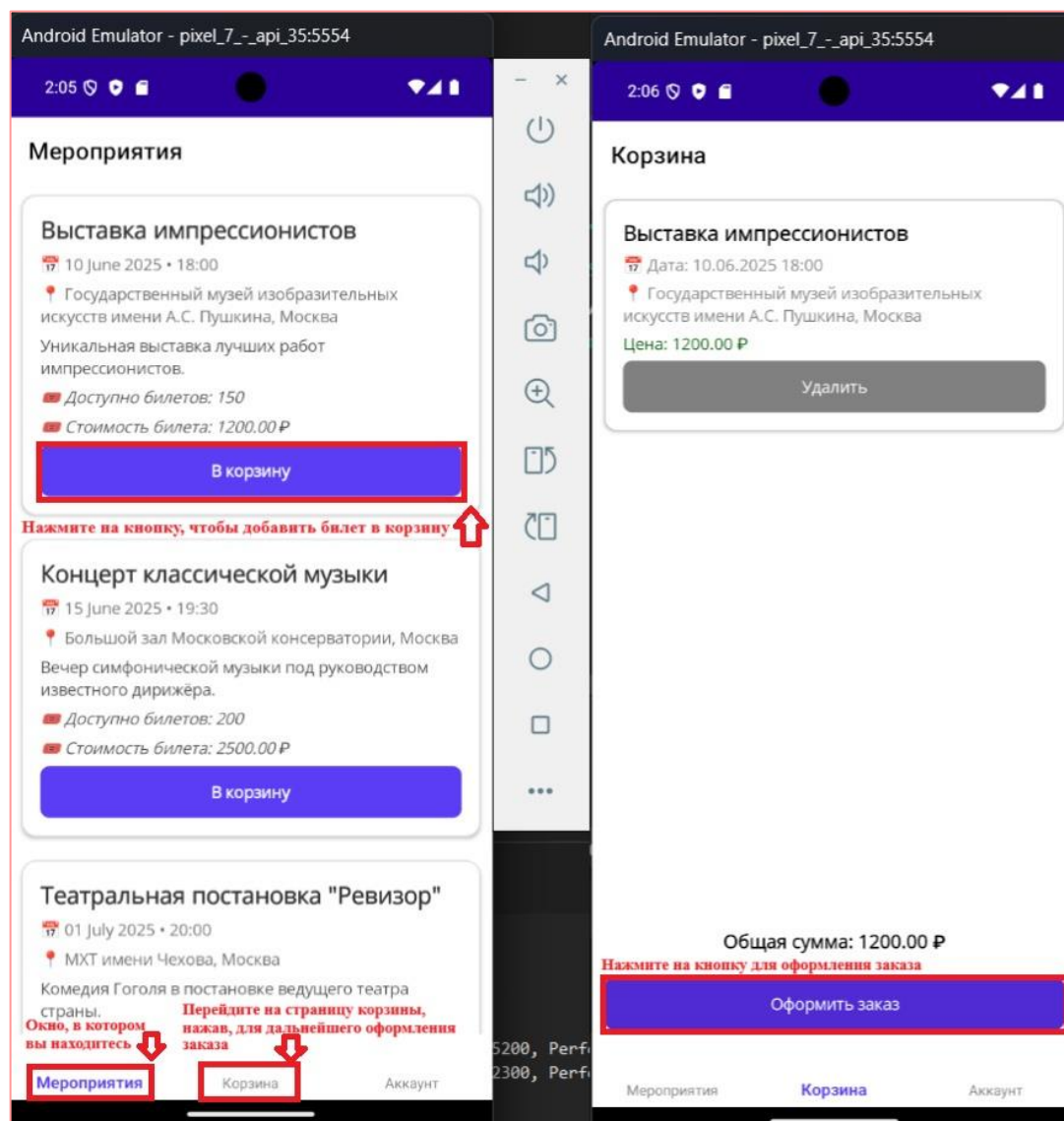
После входа откроется главный экран с списком доступных мероприятий.

На карточке мероприятия, чтобы увидеть подробную информацию:

- Название
- Дата и время
- Место проведения
- Описание
- Цена билетов
- Кол-во доступных билетов

4. Бронирование билетов

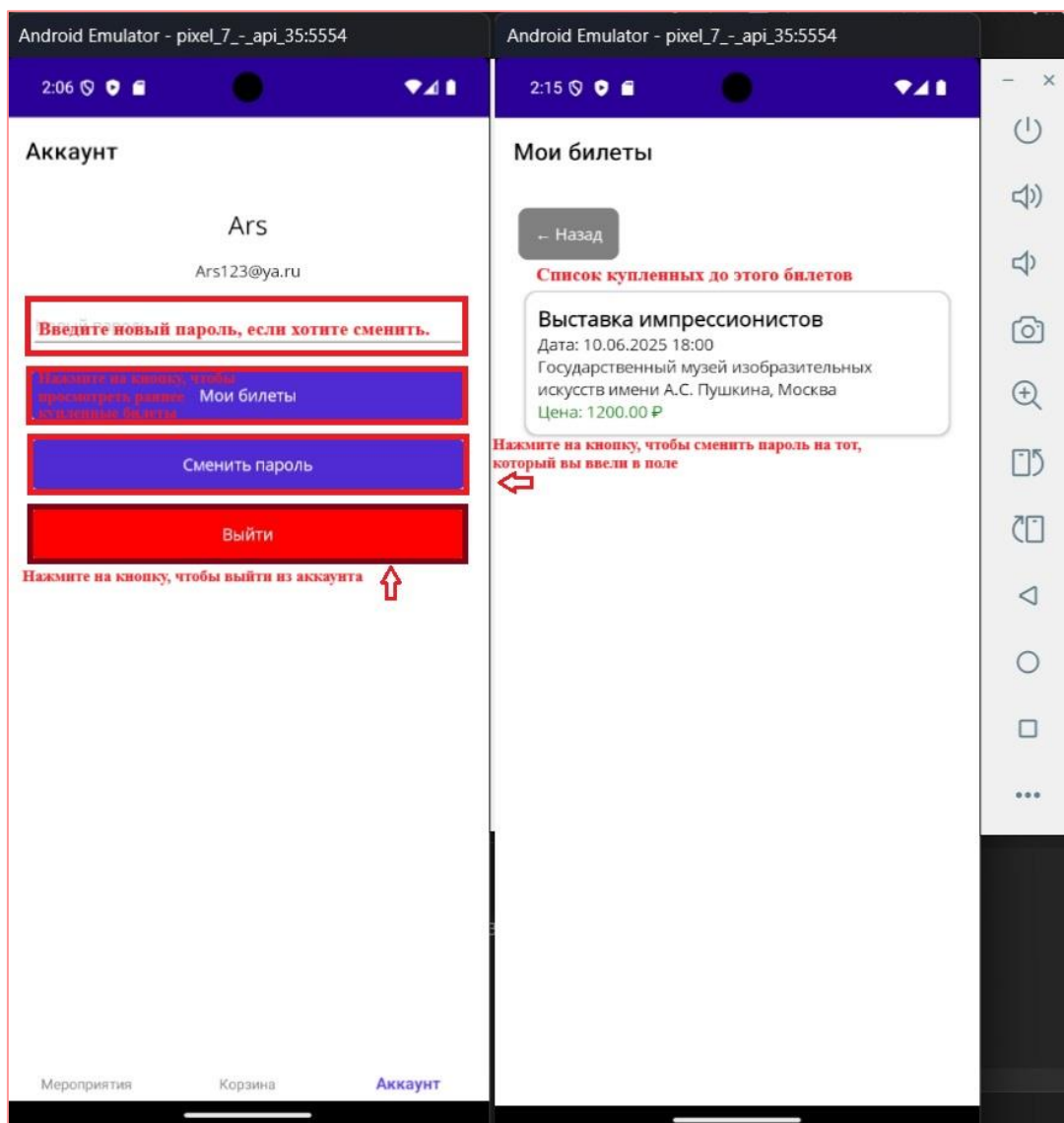
На странице мероприятия нажмите «Добавить в корзину». Перейдите в раздел «Корзина» (надпись корзина в нижнем меню). Проверьте состав заказа: Нажмите «Оформить заказ». Выберите способ оплаты и подтвердите покупку.



5. Личный кабинет (Аккаунт)

В разделе «Аккаунт» доступны:

- История заказов – просмотр всех купленных билетов.
- Смена пароля – введите новый пароль и подтвердите изменения.
- Выход из аккаунта – нажмите «Выйти» для завершения сеанса.



6. Выход из приложения

Чтобы закрыть приложение, сверните его стандартным способом для вашего устройства.